

**UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY**



**DATA STRUCTURE AND ALGORITHM
STRING MATCHING**

Lecturer:

**Dr. Lê Thanh Tùng
Mr. Trần Hoàng Quân
Mr. Nguyễn Thanh Tình**

Class:

25C11

Group:

03

25127040 - Nguyễn Bá Duy
25127212 - Trịnh Trần Hương Mai
25127559 - Thái Mạc Tường Vi

Ho Chi Minh city, March 2026

I. TABLE OF CONTENTS

I. TABLE OF CONTENTS.....	2
II. GROUP INFORMATION AND WORK ASSIGNMENT.....	3
III. PROBLEM STATEMENT.....	4
1. Definitions and Notations.....	4
2. Exact string matching problem.....	4
IV. ALGORITHM ANALYSIS.....	5
1. The Brute-Force approach.....	5
2. The Rabin-Karp algorithm.....	10
3. The Knuth-Morris-Pratt (KMP) algorithm.....	23
4. The Boyer-Moore algorithm.....	36
5. Aho-Corasick.....	49
V. EXPERIMENTAL RESULTS.....	62
1. Random data.....	62
2. DNA data.....	66
3. Special data.....	70
VI. PROJECT ORGANIZATION AND PROGRAMMING NOTES.....	71
1. Project organization.....	71
2. Compile and run environment.....	72
VII. REFERENCES.....	73
VIII. ACKNOWLEDGEMENT.....	74

II. GROUP INFORMATION AND WORK ASSIGNMENT

Student name	Student ID	Assigned Tasks	% Completed
Nguyễn Bá Duy	25127040	Implement and report on the KMP and Aho-Corasick algorithms. Organize project structure and implement auxiliary files. Analyze and provide insights on the execution time results.	100%
Trịnh Trần Hương Mai	25127212	Implement and report on the Boyer-Moore algorithm. Generate Random, DNA, and Special datasets. Measure the execution time of algorithms across various datasets.	100%
Thái Mạc Tường Vi	25127559	Implement and report on the Brute Force and Rabin-Karp algorithms. Generate charts based on the execution time of the algorithms. Record and edit the project introduction video.	100%
Total Group Completion			100%

III. PROBLEM STATEMENT

1. Definitions and Notations [1]

- A string S is a sequence of characters (Note: 0-based indexing will be used consistently throughout this report).
- $|S|$ denotes the length of the string S (often represented by variables such as n , m , etc.).
- $S[i]$ denotes the i -th character of S ($0 \leq i < |S|$).
- $S[i..j]$ denotes the substring consisting of consecutive characters from position i to position j in S ($0 \leq i \leq j < |S|$).
- The strings $S[0..i]$ ($0 \leq i < |S|$) are called prefixes of S (denoted by S').
- The strings $S[i..|S| - 1]$ are called suffixes of S .
- Σ denotes the alphabet - a finite set of characters that may appear in S ('a'-'z'). The set of all possible finite-length strings formed from the characters in Σ is denoted as Σ^* (Sigma star).

2. Exact string matching problem

Formal Definition: Given strings T and P over the alphabet Σ ($|P| \leq |T|$), find all shifts s ($0 \leq s \leq |T| - |P|$) such that $T[i + s] = P[i]$ for all $0 \leq i < |P|$.

Consider an example where the text T is "aaabbbxyzbbccccc" and we search for the pattern $P = "xyz"$. The pattern P occurs in T starting from index 6 to index 8. Therefore, we say that the pattern P occurs in the text T with a shift $s = 6$.

IV. ALGORITHM ANALYSIS

1. The Brute-Force approach

1.1. Introduction & Application

1.1.1. Introduction

The Brute-Force algorithm (also known as Naïve String Matching) is the simplest and most intuitive approach to solving the string matching problem. The main idea of the algorithm is to try all possible positions of the pattern in the text and check for matches directly. Specifically:

- Iterate through every possible starting position of P in T (a total of $n - m + 1$ positions).
- At each position, compare each character of the P with the corresponding character in T .
- If a mismatch occurs, stop the comparison at that position and turn to the next iteration.
- If all m characters match, we conclude that an occurrence of P is found.

In essence, this algorithm does not utilize any information from previous comparisons. This leads to many redundant comparisons, especially when the data contains repetitive structures. The number of character comparisons in the worst case can be very large, since nearly the entire pattern may be compared at each shift. However, the algorithm has the advantage of being easy to implement, easy to understand, and requiring no complex auxiliary data structures [1].

1.1.2. Application

Although it does not have optimal complexity in the worst case, Brute-Force is still widely used in practice due to its simplicity and effectiveness in certain scenarios.

In summary, Brute-Force is a fundamental algorithm: easy to understand, easy to implement, suitable for verifying correctness, and can serve as a baseline for performance comparison with more advanced algorithms.

1.2. Pseudocode

```
1 int m = P.length
2 int n = T.length
3 for j = 0 to (n - m)
4     bool isAppear = true
5     for k = 0 to (m - 1)
6         if (P[k] != T[j + k])
```

```

7         isAppear = false
8         //If any character is not similar, the P doesn't
          occur at this position in T
9         break
10        //After the for k loop, check if the boolean isAppear is still
          true or not
11        if (isAppear)
12            P is occur in T at position j

```

1.3. Step-by-step trace

Consider the input: $T = \text{"ababaca"}$ and $P = \text{"abac"}$. Here:

- $n = 7$ (line 2)
- $m = 4$ (line 1)
- Possible positions to shift: $n - m = 3 \rightarrow j = 0, 1, 2, 3$ (line3)
 - **At $j = 0$**

Compare $P[0..3]$ with $T[0..3]$ in the **for** k loop (line 5).

k	0	1	2	3
$P[k]$	a	b	a	c
$T[j + k]$	a	b	a	b
isAppear	true	true	true	false $\rightarrow$ break

$\text{isAppear} = \text{false} \rightarrow P$ does NOT appear at position $j = 0$.

- **At $j = 1$**

Compare $P[0..3]$ with $T[1..4]$ in the **for** k loop (line 5).

k	0	1	2	3
$P[k]$	a	b	a	c
$T[j + k]$	b	a	b	a
isAppear	false $\rightarrow$ break			

$\text{isAppear} = \text{false} \rightarrow P$ does NOT appear at position $j = 1$.

- At $j = 2$

Compare $P[0..3]$ with $T[2..5]$ in the **for** k loop (line 5).

k	0	1	2	3
$P[k]$	a	b	a	c
$T[j + k]$	a	b	a	c
isAppear	true	true	true	true

isAppear = true $\rightarrow$ P appears at position $j = 2$.

- At $j = 3$

Compare $P[0..3]$ with $T[3..6]$ in the **for** k loop (line 5).

k	0	1	2	3
$P[k]$	a	b	a	c
$T[j + k]$	b	a	c	a
isAppear	false $\rightarrow$ break			

isAppear = false $\rightarrow$ P does NOT appear at position $j = 3$.

1.4. Complexity

1.4.1. Time complexity

Because this is an algorithm with a simple approach, it does not have a preprocessing phase but only a matching phase. Therefore, the time complexity is entirely based on the complexity of the matching phase.

According to the pseudocode, the algorithm has 2 nested loops: **for** $j = 0$ to $(n - m)$ (line 3) and **for** $k = 0$ to $(m - 1)$ (line 5). In the case of a full check, the program will have to perform a maximum of m comparisons for each value of j (the **for** j loop runs $n - m + 1$ times). In general, the maximum number of comparisons is approximately:

$$m * (n - m + 1)$$

If $m \ll n$, we can approximate this to:

$$\sim m * n$$

Based on the analysis above, the time complexity for this algorithm is [1]:

$$O(m * (n - m + 1)) \text{ or } \sim O(m * n)$$

1.4.2. Space complexity

According to the pseudocode, the algorithm only uses a few auxiliary variables such as indices and a boolean flag (`isAppear`). Therefore, the required extra memory is constant $\rightarrow$ Space complexity = $O(1)$.

1.5. Discussion

1.5.1. Best-case scenario

The best-case scenario occurs when at each shift position j , the first character of the P is already different from the corresponding character in T . In this case:

- The **for** k loop only runs 1 time and then breaks.
- The total number of comparisons is approximately $n - m + 1$.

In this situation, the algorithm has a near-linear performance relative to the text length. This usually happens when:

- The P has a distinctly different first character compared to most characters in T .
- T data is highly random and does not repeat much.

1.5.2. Worst-case scenario

Contrary to the best-case, the worst-case scenario happens when T and P have a repeating structure that causes many initial characters to match before a mismatch occurs.

For example [2]:

T : "AAAAAAAAB"

P : "AAAAB"

At each shift position:

The first 4 characters "AAAA" match.

A mismatch occurs at the last character "B".

As a result, the algorithm nearly has to compare all m characters at every position. The total number of comparisons is approximately $m * n$. In this case, the algorithm becomes inefficient due to performing many repetitive comparisons.

1.5.3. When to use the Brute-Force algorithm

Brute-Force is suitable when:

- The text data size is small.
- The pattern length is short.
- A baseline algorithm is needed to verify correctness before implementing more complex algorithms.
- Memory usage needs to be optimized.

Brute-Force is not suitable when:

- The text size is large.
- There are many keywords to search for.
- The data is highly repetitive, causing many redundant comparisons.
- High and stable performance is required across all scenarios.

In these situations, algorithms like KMP or Boyer-Moore (which we will introduce to you in the following sections) will provide better and more stable performance.

2. The Rabin-Karp algorithm

2.1. Introduction & Application

2.1.1. Introduction

The Rabin–Karp algorithm is a string matching method based on hashing techniques to reduce the number of direct character comparisons compared to Brute-Force. The core idea of the algorithm is:

- Compute the hash value of the P and the hash value of each substring of length m in T.
- Compare the two hash values.
- If the hash values are different → definitely not a match.
- If the hash values are equal → perform character-by-character comparison to confirm (to avoid false matches due to hash collisions).

Rabin–Karp uses the idea of representing a string as a number in a BASE number system and taking modulo MOD to avoid overflow. Characters are mapped to corresponding numeric values to ensure correctness in arithmetic, and BASE must always be greater than the number of distinct characters that can appear in T or P. The hash formula is [1]:

$$\text{HashP} = P[m-1] + \text{BASE} * (P[m-2] + \text{BASE} * (P[m-3] + \dots + \text{BASE} * (P[1] + \text{BASE} * P[0]) \dots))$$

Rabin–Karp applies the rolling hash technique to compute the hash of the next substring in constant time. When shifting from position $j - 1$ to j , instead of recomputing from scratch, we update using [1]:

$$\text{HashT}_j = \text{BASE} * (\text{HashT}_{j-1} - \text{BASE}^{m-1} * T[j-m]) + T[j]$$

This approach significantly reduces the number of direct character comparisons compared to Brute-Force. Therefore, Rabin-Karp is an efficient method when you need to find multiple patterns or when you want to reduce character comparisons by checking hashes first.

2.1.2. Application

The Rabin–Karp algorithm is widely used in real-world string searching problems, especially when processing large text data or multiple patterns simultaneously. Thanks to hashing and rolling hash, it can quickly eliminate mismatched positions without comparing characters one by one.

A common application is plagiarism detection systems. T is divided into small segments and each segment is converted into a hash value. Rabin–Karp is also used in searching within text databases, system logs, or biological data such as DNA sequences. In these cases, the data is often very large, and updating hash values in $O(1)$ time while sliding the window helps the algorithm remain efficient. The algorithm can also be extended to

handle multiple patterns by storing pattern hashes in a hash table, significantly improving performance when the number of patterns is large.

Overall, Rabin–Karp is suitable for systems that require fast searching on large datasets, demand high performance, accept a very small probability of hash collision, and include a verification step to ensure correctness.

2.2. Pseudocode

It is agreed that the following arithmetic operations (addition, subtraction, multiplication) will be performed using the modulo operation $\text{MOD} = 2147483647$ after calculation. For example: $250 * 97$ represents $(250 * 97) \% \text{MOD}$; $12 - 345$ represents $(12 - 345 + \text{MOD}) \% \text{MOD}$.

```
1 //Initialize a function that type casts a character to an integer,
  and we define that 'a' is equal to 1.
2 int char_to_int(char c)
3     return (c - 'a' + 1)

4 int BASE = 29
5 int m = P.length
6 int n = T.length

7 //Check if the P is shorter than T
8 if (m <= n)
9     long long TextHash = 0
10    long long PatternHash = 0
11    //Compute the hash of P
12    for i = 0 to (m - 1)
13        PatternHash = PatternHash * BASE + char_to_int(P[i])

14    //Compute the hash of T and compare the two hashes to
    find their position.
15    for j = 0 to (n - 1)
16        bool isAppear = false
17        if (j < m)
18            //Add low-order numbers since its length is equal
```

```

    to m
19     TextHash = TextHash * BASE + char_to_int(T[j])
20     if (j < m - 1) continue
21     //If the text hash hasn't included enough string
       segments equal to the P length, then no
       comparison will be made.
22     else
23         //Calculate the hash at j by removing the
       high-order of hash at (j - 1) and adding the
       low-order at position j.
24         int highOrder = char_to_int(T[j - m])
25         int lowOrder = char_to_int(T[j])
26         TextHash = TextHash - highOrder * BASEm - 1
27         TextHash = TextHash * BASE + lowOrder

28     //Compare hashes
29     if (TextHash == PatternHash)
30         isAppear = true
31         for k = 0 to (m - 1)
32             //Increase the variable that counts
               comparisons of characters here
33             if (P[k] != T[j - m + 1 + k])
34                 isAppear = false
35                 //If any character is not similar, the
                   P doesn't occur at this position in T
36                 break
37     //After the for k loop, check if boolean isAppear is still
       true or not
38     if(isAppear)
39         P is occur in T at position j
```

2.3. Step-by-step trace

2.3.1. Rationale for Choosing BASE and MOD

Before presenting the detailed execution trace of the Rabin–Karp algorithm, it is necessary to justify the selection of the two fundamental constants used in the hash computation:

- **BASE = 29**
- **MOD = 2147483647**

These two constants directly affect the correctness, efficiency, and collision probability of the rolling hash function.

2.3.1.1. Role of BASE in Polynomial Hashing

In the Rabin-Karp algorithm, a string of length m is treated as a polynomial with a chosen base:

$$\text{HashP} = P[m-1] + \text{BASE} * (P[m-2] + \text{BASE} * (P[m-3] + \dots + \text{BASE} * (P[1] + \text{BASE} * P[0]) \dots))$$

Therefore, BASE acts as a radix, similar to the positional numeral system in mathematics. The purpose of selecting a suitable BASE is to ensure that the position of each character significantly affects the hash value and to avoid collisions caused by character permutations (for example, "abc" and "cba" must not produce the same hash value).

So why choose BASE = 29?

Within the scope of this project, the input data only consists of English lowercase letters (a–z). Therefore, BASE must be greater than the alphabet size = 26 [1], [2]:

$$\text{BASE} = 29$$

In this case:

- $29 > 26$, ensuring each character's value is always smaller than BASE.
- 29 is a prime number, which helps improve distribution when performing modulo operations.
- 29 is small enough to limit the risk of integer overflow before performing the mod operation.

Thus, BASE = 29 is a well-balanced choice between correctness, efficiency, and reducing the collision probability mentioned above.

2.3.1.2. Role of MOD in Hash Computation

If the modulo operation is not used, the polynomial hash value will increase exponentially by BASE^m . With a large m , this value will exceed the storage capacity of standard integer (int) type. Therefore, we must use the modulo operation to [1]:

- Prevent integer overflow.
- Keep the hash value within a fixed range $0 \leq H(s) < \text{MOD}$.

So why choose MOD = 2147483647?

Because this value:

- Is a Mersenne prime number.
 - Choosing a prime number for MOD ensures that hash values are evenly distributed across the $[0, \text{MOD}-1]$ space, thereby reducing the probability of collisions. Additionally, in the field of integers modulo a prime, arithmetic operations possess better algebraic properties, which helps maintain the distinctness between different strings [1].
 - Furthermore, choosing a Mersenne prime (in the form $2^x - 1$) for MOD optimizes the modulo calculation by allowing bitwise operations (bitwise AND and shift) instead of standard division. This helps improve performance during computation.
- It is the largest prime number within the signed 32-bit integer range.
 - It is small enough to be safely stored in a long long (64-bit) type → any multiplication between two numbers smaller than MOD is safe within the long long range limits.

2.3.1.3. Collision Probability Analysis

Analysis Assuming the hash values are evenly distributed within the range $[0, \text{MOD}-1]$, the probability that two different strings result in the same hash value is [1]:

$$P = 1 / \text{MOD}$$

$$\Leftrightarrow P = 1 / 2147483647 \sim 4,6 \cdot 10^{-10}$$

This is an extremely small probability, and even if it occurs, the impact is insignificant.

2.3.2. Trace

Consider the input: T = "agvzpwmkdefinedb" and P = "defined". Here:

- $n = 16$ (line 6)
- $m = 7$ (line 5)
- $m < n \rightarrow$ we can proceed with checking (line 8)
- TextHash = 0 (line 9)
- PatternHash = 0 (line 10)

2.3.2.1. Preprocessing

- **Compute the hash of P**

According to the formula at line 13: $\text{PatternHash} = \text{PatternHash} * \text{BASE} + \text{char_to_int}(P[i])$.

The number of possible shift positions: $(m - 1) + 1 = 7 \rightarrow i = 0, 1, 2, 3, 4, 5, 6$ (line 12)

i	0	1	2	3	4	5	6
P[i]	d	e	f	i	n	e	d
value P[i]	4	5	6	9	14	5	4
PatternHash	4	121	3 515	101 944	2 956 390	85 735 315	338 840 492

- **Compute the first window of T**

According to the formula at line 19: $\text{TextHash} = \text{TextHash} * \text{BASE} + \text{char_to_int}(T[j])$.

The number of possible shift positions based on lines 15 and 17 (conditional constraints): $(m - 1) + 1 = 7 \rightarrow j = 0, 1, 2, 3, 4, 5, 6$.

The next action depends on the condition in the **if** statement (line 20).

j	0	1	2	3	4	5	6
T[j]	a	g	v	z	p	w	m
value T[j]	1	7	22	26	16	23	13
TextHash	1	36	1 066	30 940	897 276	26 021 027	754 609 796
Next action	continue	continue	continue	continue	continue	continue	go to line 29

- **Matching**

This phase begins when variable j from the **for** loop (starting at line 15) reaches $j = (m-1)$ and the Hash value calculation at the **if** block (line 17) is complete. At this point, we proceed to the comparison and matching commands (lines 28, 38) because the value $j = (m-1)$ bypasses the **if** condition at line 20.

- **At $j = 6$**

At line 29, only compare PatternHash and TextHash:

$$338\ 840\ 492 \neq 754\ 609\ 796$$

$\rightarrow \text{isAppear} = \text{false} \rightarrow P$ does not appear at this position.

○ At $j = 7$

Index	$j - m$							j	
P	X	d	e	f	i	n	e	d	X
T	a	g	v	z	p	w	m	k	...
Value of $T[\text{index}]$	1	7	22	26	16	23	13	11	

$\text{highOrder} = \text{char_to_int}(T[j - m]) = 1$

$\text{lowOrder} = \text{char_to_int}(T[j]) = 11$

$\text{TextHash} = (\text{TextHash} - \text{highOrder} * \text{BASE}^{m-1}) * \text{BASE} + \text{lowOrder}$
 $= 338\ 840\ 492$

Compare PatternHash and TextHash:

$338\ 840\ 492 == 338\ 840\ 492$

When the 2 hashes are equal, perform a character-by-character comparison between the 2 strings in the **for** k loop at line 30:

k	0	1	2	3	4	5	6
$j - m + 1 + k$	1	2	3	4	5	6	7
P	d	e	f	i	n	e	d
T	g	v	z	p	w	m	k
isAppear	false → break						

→ isAppear = false → P does not appear at this position.

○ At $j = 8$

Index		$j - m$							j	
P	X	X	d	e	f	i	n	e	d	X
T	...	g	v	z	p	w	m	k	d	...
Value of $T[\text{index}]$		7	22	26	16	23	13	11	4	

$\text{highOrder} = \text{char_to_int}(T[j - m]) = 7$

$\text{lowOrder} = \text{char_to_int}(T[j]) = 4$

$\text{TextHash} = (\text{TextHash} - \text{highOrder} * \text{BASE}^{m-1}) * \text{BASE} + \text{lowOrder}$
 $= 746\ 389\ 753$

Compare PatternHash with TextHash:

$338\ 840\ 492 != 746\ 389\ 753$

→ isAppear = false → P does not appear at this position.

○ At j = 9

Index		j - m							j	
P	X	X	d	e	f	i	n	e	d	X
T	...	v	z	p	w	m	k	d	e	...
Value of T[index]		22	26	16	23	13	11	4	5	

highOrder = **char_to_int**(T[j - m]) = 22

lowOrder = **char_to_int**(T[j]) = 5

TextHash = (TextHash - highOrder * BASE^{m-1}) * BASE + lowOrder
= 777 793 093

Compare PatternHash with TextHash:

338 840 492 != 777 793 093

→ isAppear = false → P does not appear at this position.

○ At j = 10

Index		j - m							j	
P	X	X	d	e	f	i	n	e	d	X
T	...	z	p	w	m	k	d	e	f	...
Value of T[index]		26	16	23	13	11	4	5	6	

highOrder = **char_to_int**(T[j - m]) = 26

lowOrder = **char_to_int**(T[j]) = 6

TextHash = (TextHash - highOrder * BASE^{m-1}) * BASE + lowOrder
= 1 408 461 422

Compare PatternHash with TextHash:

338 840 492 != 1 408 461 422

→ isAppear = false → P does not appear at this position.

○ At j = 11

Index		j - m							j	
P	X	X	d	e	f	i	n	e	d	X

T	...	p	w	m	k	d	e	f	i	...
Value of T[index]		16	23	13	11	4	5	6	9	

highOrder = **char_to_int**(T[j - m]) = 16

lowOrder = **char_to_int**(T[j]) = 9

TextHash = (TextHash - highOrder * BASE^{m-1}) * BASE + lowOrder
= 1 070 561 473

Compare PatternHash with TextHash:

338 840 492 != 1 070 561 473

→ isAppear = false → P does not appear at this position.

○ At j = 12

Index		j - m							j	
P	X	X	d	e	f	i	n	e	d	X
T	...	w	m	k	d	e	f	i	n	...
Value of T[index]		23	13	11	4	5	6	9	14	

highOrder = **char_to_int**(T[j - m]) = 23

lowOrder = **char_to_int**(T[j]) = 14

TextHash = (TextHash - highOrder * BASE^{m-1}) * BASE + lowOrder
= 1 518 831 261

Compare PatternHash with TextHash:

338 840 492 != 1 518 831 261

→ isAppear = false → P does not appear at this position.

○ At j = 13

Index		j - m							j	
P	X	X	d	e	f	i	n	e	d	X
T	...	m	k	d	e	f	i	n	e	...
Value of T[index]		13	11	4	5	6	9	14	5	

highOrder = **char_to_int**(T[j - m]) = 13

lowOrder = **char_to_int**(T[j]) = 5

$$\begin{aligned}\text{TextHash} &= (\text{TextHash} - \text{highOrder} * \text{BASE}^{m-1}) * \text{BASE} + \text{lowOrder} \\ &= 186\ 340\ 905\end{aligned}$$

Compare PatternHash with TextHash:

$$338\ 840\ 492 \neq 186\ 340\ 905$$

→ isAppear = false → P does not appear at this position.

○ At j = 14

Index		j - m							j	
P	X	X	d	e	f	i	n	e	d	X
T	...	k	d	e	f	i	n	e	d	...
Value of T[index]		11	4	5	6	9	14	5	4	

$$\text{highOrder} = \text{char_to_int}(T[j - m]) = 11$$

$$\text{lowOrder} = \text{char_to_int}(T[j]) = 4$$

$$\begin{aligned}\text{TextHash} &= (\text{TextHash} - \text{highOrder} * \text{BASE}^{m-1}) * \text{BASE} + \text{lowOrder} \\ &= 338\ 840\ 492\end{aligned}$$

Compare PatternHash with TextHash:

$$338\ 840\ 492 == 338\ 840\ 492$$

When the 2 hashes are equal, perform a character-by-character comparison between the 2 strings in the **for** k loop at line 30:

k	0	1	2	3	4	5	6
j - m + 1 + k	8	9	10	11	12	13	14
P	d	e	f	i	n	e	d
T	d	e	f	i	n	e	d
isAppear	true	true	true	true	true	true	true

→ isAppear = true → P appears at this position.

○ At j = 15

Index		j - m							j	
P	X	X	d	e	f	i	n	e	d	X
T	...	d	e	f	i	n	e	d	b	...
Value of T[index]		4	5	6	9	14	5	4	2	

```

highOrder = char_to_int(T[j - m]) = 4
lowOrder = char_to_int(T[j]) = 2
TextHash = (TextHash - highOrder * BASEm - 1) * BASE + lowOrder
           = 956 411 150

```

Compare PatternHash with TextHash:

```
338 840 492 != 956 411 150
```

→ isAppear = false → P does not appear at this position.

2.4. Complexity

2.4.1. Time complexity

2.4.1.1. Preprocessing time

According to the pseudocode, the algorithm has 3 steps in the preprocessing phase:

- Compute the hash of the P segment: **for** $i = 0$ to $(m - 1)$ (line12) → Complexity is $O(m)$.
- Compute the initial hash of T segment: **for** $i = 0$ to $(n - 1)$ (line15) and **if** $(i < m)$ (line17) → Complexity is $O(m)$.
- Compute the value BASE^{m-1} → Complexity is $O(m - 1)$.

Overall, the complexity is: $O(m) + O(m) + O(m - 1) = O(m)$.

2.4.1.2. Matching time

According to the pseudocode, during the matching phase, the algorithm simultaneously updates the rolling hash and performs comparisons while traversing every window position in T (from $m - 1$ to $n - 1$). With each traversal step, executing the rolling hash calculations only takes constant time. In cases where the 2 hashes match, the program must further inspect each character in the **for** $k = 0$ to $(m - 1)$ (line 30). Thus, the general time complexity is:

$$O((n - m + 1) * m) \sim O(n * m)$$

2.4.1.3. Time complexity

Based on the analysis above, the total time complexity of the Rabin-Karp algorithm is $O(n * m)$ [2].

2.4.2. Space complexity

According to the pseudocode, we use:

- 2 variables to store hashes (TextHash, PatternHash).

- 1 variable to store BASE^{m-1} .
- Several auxiliary variables (highOrder, lowOrder, isAppear).

Therefore, the space complexity is: $O(1)$.

2.5. Discussion

2.5.1. Best-Case Scenario

The best-case scenario occurs when:

- There are no hash collisions.
- If there are hash collisions:
 - Either the number of identical hashes is extremely small.
 - Or when hashes do match, the character differs right away at $k = 0$.

In this case:

- The rolling hash runs in $O(n)$.
- The **for** k loop for character-by-character checking is almost negligible.

The complexity in this case is approximately $O(n)$.

2.5.2. Average-Case Scenario

In reality, the hashes of character strings are distributed fairly evenly: $(n * m) / \text{MOD}$. Because the probability for 2 different random strings to have colliding hashes is: $1 / \text{MOD}$. With the inputs Row and Col ≤ 1000 , we have: $1000000 / 2147483647 \sim 4,6 \cdot 10^{-4} \rightarrow$ very small. The complexity in average cases is approximately $O(n)$, meaning Rabin-Karp operates in near-linear time in most scenarios [1], [2].

2.5.3. Worst-Case Scenario

The worst-case scenario occurs when:

- The hash of every window is equal to PatternHash.
- The strings have many matching characters.

In this case, every single position out of the $(n - m + 1)$ possibilities must run the **for** k (line 30). Hence, the complexity can degrade to $O((n - m + 1) * m)$. This complexity is equivalent to the Brute-Force algorithm.

2.5.4. When to use the Rabin - Karp algorithm

Rabin-Karp is suitable when:

- *Needing to find multiple patterns in the same text:* When there are many patterns to search for, we can pre-compute the hashes for all patterns and then quickly compare them against the rolling hash of the text.

- *The text data is large and you need to minimize character comparisons:* With a massive input size, Rabin-Karp can quickly filter out mismatched positions using hashes instead of direct character-by-character checking at every position like Brute-Force. If the collision probability is low, the algorithm's efficiency will be close to $O(n)$.
- *Processing continuous (streaming) string data:* The rolling hash helps us continuously and seamlessly update the Hash by dropping the high-order term and adding the low-order term at each window shift. Therefore, Rabin-Karp is highly suitable for finding substrings, processing DNA sequences, or detecting content duplication.

Rabin-Karp is not suitable when:

- *Hash collisions can occur frequently:* If the algorithm sets MOD as a small integer or a non-prime number, the collision probability between 2 distinct strings will increase. Alternatively, when the text data contains many identical repeating strings, hash collisions surge, driving the complexity closer to $O(n * m)$. At this point, Rabin-Karp will perform almost identically to Brute-Force.
- *Absolute accuracy is required and you do not want to double-check:* If the system requirements dictate that a word-by-word verification step should be avoided, Rabin-Karp is inappropriate because the probability of 2 hashes colliding always exists, even though it is extremely small.
- *The environment does not support large integers:* Because of the nature of the algorithm, you must select two suitable prime numbers for BASE and MOD. If the input involves many different character types, BASE and MOD must be scaled up to very large prime numbers to ensure the hash collision probability remains manageable. In environments systems that do not support storing such massive integers, the Rabin-Karp algorithm will become inefficient.

3. The Knuth-Morris-Pratt (KMP) algorithm

3.1. Introduction & Application

3.1.1. Introduction

- The main idea of the KMP algorithm is to reduce the number of comparisons between the original string (called the *Text*, denoted by T) and the string to be searched (called the *Pattern*, denoted by P) [2].
- To achieve this, we construct an array called LPS (Longest Prefix Suffix) based on the pattern P , where $LPS[j]$ is the length of the longest substring P' such that it is both a proper prefix and a suffix of the substring $P[0..j]$ [1]. If $LPS[j] = k$, then $P' = P[0..k-1] = P[j-k+1..j]$. Note: For every j , $LPS[j] \leq j$; therefore, $LPS[0] = 0$.
- Let $match[i]$ denote the length of the longest substring P' of P such that P' is both a suffix of $T[0..i]$ and a prefix of P . If P appears in T , then $P' = P$ is a suffix of $T[0..i]$, and $match[i] = m$. In practice, there's no need to store this array; a single variable will suffice. However, to ensure clarity and enhance understanding, other sections will continue to utilize the match array for explanatory purposes, while the pseudocode employs the variable k as a more practical alternative. [1].
- Using the LPS array, whenever a mismatch occurs, instead of restarting the comparison from the beginning, we can determine values k (possibly equal to 0) such that $T[i-k..i-1] = P[0..k-1]$ [1].
- At this point, we only need to compare $T[i]$ and $P[k]$, for each $k > 0$:
 - (1) If $T[i] = P[k]$, then $match[i] = k + 1$ [1].
 - (2) If $T[i] \neq P[k]$, we update $k = LPS[k-1]$ and continue comparing until a match is found or $k = 0$ [1].
- When $k = 0$: $match[i] = 1$ if $T[i] = P[0]$, or $match[i] = 0$ if $T[i] \neq P[0]$ [1].
- For every $match[i] = m$, the string P appears at positions $i - m + 1..i$ in the string T

3.1.2. Application

In practice, the KMP algorithm is widely applied in text editors (search functionality, enabling accurate character matching), plagiarism detection (measuring the similarity between documents), spam detection systems, and search engines. It is also used in bioinformatics for matching DNA sequences [3].

3.2. Pseudocode [4]

```

1  //Build LPS array
2  let LPS[0..m - 1] be a new array //m is size of LPS
3  LPS[0] = 0
4  k = 0
5  for j: 1 → m - 1
6      while k > 0 and P[k] ≠ P[j]
7          k = LPS[k - 1]
8      if P[k] == P[j]
9          k = k + 1
10     LPS[j] = k

11 //Search phase
12 k = 0
13 for i: 0 → n - 1
14     while k > 0 and T[i] ≠ P[k]
15         k = LPS[k - 1]
16     if T[i] == P[k]
17         k = k + 1
18     if k == m
19         print "Pattern occurs with shift" i - m + 1
20         k = LPS[k - 1]

```

3.3. Step-by-step trace

- Given the text string $T = \text{"ABBCBABACBABA"}$ and the pattern string $P = \text{"BABAC"}$.

3.3.1. Build LPS array

- Step 1: Initialize the LPS array with 5 elements.
- Step 2: Set $LPS[0] = 0$.

Index	0	1	2	3	4
Pattern	B	A	B	A	C
LPS[i]	0				

- Step 3: Initialize $k = 0$, representing the length of the longest proper prefix that is also a suffix of P.
- Step 4: Initialize $j = 1$ as the loop index.
- Step 5: Consider $j = 1$.
 Since $k = 0$,
 → Do not decrease k .
 Compare $P[k]$ and $P[j]$: 'B' $\neq$ 'A'.
 → Do not increase k .
 Set $LPS[j] = k = 0$.
 Increment j by 1 ($j = 2$).

Index	0	1	2	3	4
Pattern	B	A	B	A	C
LPS[i]	0	0			

- Step 6: Consider $j = 2$ (k is still 0).
 Since $k = 0$,
 → Do not decrease k .
 Compare $P[k]$ and $P[j]$: 'B' = 'B'.
 → Increment k by 1 ($k = 1$).
 Set $LPS[j] = k = 1$.
 Increment j by 1 ($j = 3$).

Index	0	1	2	3	4
Pattern	B	A	B	A	C
LPS[i]	0	0	1		

- Step 7: Consider $j = 3$ ($k = 1$).
 Since $k > 0$ and $P[k] = P[j]$,
 → Do not decrease k .
 Compare $P[k]$ and $P[j]$: 'A' = 'A'.
 → Increment k by 1 ($k = 2$).
 Set $LPS[j] = k = 2$.
 Increment j by 1 ($j = 4$).

Index	0	1	2	3	4
Pattern	B	A	B	A	C
LPS[i]	0	0	1	2	

- Step 8: Consider $j = 4$ ($k = 2$).

Since $k > 0$ and $P[k] \neq P[j]$ (the condition is satisfied),

→ Decrease k : $k = \text{LPS}[k - 1] = 0$.

Index	0	1	2	3	4
Pattern	B	A	B	A	C
LPS[i]	0	0	1	2	

At this point, since $k = 0$,

→ Do not continue decreasing k .

Compare $P[k]$ and $P[j]$: 'B' $\neq$ 'C'.

→ Do not increase k .

Set $\text{LPS}[j] = k = 0$.

Increment j by 1 ($j = 5$).

→ End of the loop.

Index	0	1	2	3	4
Pattern	B	A	B	A	C
LPS[i]	0	0	1	2	0

3.3.2. Matching phase

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Text	A	B	B	C	B	A	B	A	C	B	A	B	A
Pattern	B	A	B	A	C								
LPS[i]	0	0	1	2	0								

match[i]													
----------	--	--	--	--	--	--	--	--	--	--	--	--	--

- Step 1: Initialize $i = 0$ as the loop index.
- Step 2: Initialize $k = 0$, representing the length of the longest proper prefix of P that is also a suffix of $T[0..i]$ (equivalently, $match[i]$ in the Introduction).
- Step 3: Consider $i = 0$.

Since $k = 0$,

→ Do not decrease k .

Compare $T[i]$ and $P[k]$: 'A' $\neq$ 'B'.

→ Do not increase k .

$k \neq 5$ (the length of P).

($match[0] = k = 0$)

→ Do not decrease k .

Increment i by 1 ($i = 1$).

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Text	A	B	B	C	B	A	B	A	C	B	A	B	A
Pattern	B	A	B	A	C								
LPS[i]	0	0	1	2	0								
match[i]	0												

- Step 4: Consider $i = 1$ ($k = 0$).
 Since $k = 0$,
 → Do not decrease k .
 Compare $T[i]$ and $P[k]$: 'B' = 'B'.
 → Increment k by 1 ($k = 1$).
 ($match[1] = k = 1$)
 $k \neq 5$ (the length of P).
 → Do not decrease k .
 Increment i by 1 ($i = 2$).

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Text	A	B	B	C	B	A	B	A	C	B	A	B	A
Pattern	B	A	B	A	C								
LPS[i]	0	0	1	2	0								
match[i]	0	1											

- Step 5: Consider $i = 2$ ($k = 1$).
 Since $k > 0$ and $T[i] \neq P[k]$,
 → Decrease k : $k = LPS[k - 1] = 0$.
 At this point, since $k = 0$,
 → Do not continue decreasing k .

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Text	A	B	B	C	B	A	B	A	C	B	A	B	A
Pattern	B	A	B	A	C								
LPS[i]	0	0	1	2	0								
match[i]	0	1											

Compare $T[i]$ and $P[k]$: 'B' = 'B'.

→ Increment k by 1 ($k = 1$).

($match[2] = k = 1$)

$k \neq 5$ (the length of P).

→ Do not decrease k .

Increment i by 1 ($i = 3$).

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Text	A	B	B	C	B	A	B	A	C	B	A	B	A
Pattern	B	A	B	A	C								
LPS[i]	0	0	1	2	0								
match[i]	0	1	1										

- Step 6: Consider $i = 3$ ($k = 1$).
 Since $k > 0$ and $T[i] \neq P[k]$,
 → Decrease k : $k = LPS[k - 1] = 0$.
 At this point, since $k = 0$,
 → Do not continue decreasing k .

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Text	A	B	B	C	B	A	B	A	C	B	A	B	A
Pattern	B	A	B	A	C								
LPS[i]	0	0	1	2	0								
match[i]	0	1	1										

Compare $T[i]$ and $P[k]$: 'C' $\neq$ 'B'.

→ Do not increase k .

($match[3] = k = 0$)

$k \neq 5$ (the length of P).

→ Do not decrease k .

Increment i by 1 ($i = 4$).

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Text	A	B	B	C	B	A	B	A	C	B	A	B	A
Pattern	B	A	B	A	C								
LPS[i]	0	0	1	2	0								
match[i]	0	1	1	0									

- Step 7: Consider $i = 4$ ($k = 0$).
 Since $k = 0$,
 → Do not decrease k .
 Compare $T[i]$ and $P[k]$: 'B' = 'B'.
 → Increment k by 1 ($k = 1$).
 ($match[4] = k = 1$)
 $k \neq 5$ (the length of P).

→ Do not decrease k.

Increment i by 1 (i = 5).

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Text	A	B	B	C	B	A	B	A	C	B	A	B	A
Pattern	B	A	B	A	C								
LPS[i]	0	0	1	2	0								
match[i]	0	1	1	0	1								

- Step 8: Consider i = 5 (k = 1).
 Since $k > 0$ and $T[i] = P[k]$,
 → Do not decrease k.
 Compare $T[i]$ and $P[k]$: 'A' = 'A'.
 → Increment k by 1 (k = 2).
 (match[5] = k = 2)
 $k \neq 5$ (the length of P).
 → Do not decrease k.
 Increment i by 1 (i = 6).

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Text	A	B	B	C	B	A	B	A	C	B	A	B	A
Pattern	B	A	B	A	C								
LPS[i]	0	0	1	2	0								
match[i]	0	1	1	0	1	2							

- Step 9: Consider i = 6 (k = 2).
 Since $k > 0$ and $T[i] = P[k]$,
 → Do not decrease k.
 Compare $T[i]$ and $P[k]$: 'B' = 'B'.
 → Increment k by 1 (k = 3).
 (match[6] = k = 3)
 $k \neq 5$ (the length of P).
 → Do not decrease k.

Increment i by 1 ($i = 7$).

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Text	A	B	B	C	B	A	B	A	C	B	A	B	A
Pattern	B	A	B	A	C								
LPS[i]	0	0	1	2	0								
match[i]	0	1	1	0	1	2	3						

- Step 10: Consider $i = 7$ ($k = 3$).
 Since $k > 0$ and $T[i] = P[k]$,
 → Do not decrease k .
 Compare $T[i]$ and $P[k]$: 'A' = 'A'.
 → Increment k by 1 ($k = 4$).
 ($match[7] = k = 4$)
 $k \neq 5$ (the length of P).
 → Do not decrease k .
 Increment i by 1 ($i = 8$).

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Text	A	B	B	C	B	A	B	A	C	B	A	B	A
Pattern	B	A	B	A	C								
LPS[i]	0	0	1	2	0								
match[i]	0	1	1	0	1	2	3	4					

- Step 11: Consider $i = 8$ ($k = 4$).
 Since $k > 0$ and $T[i] = P[k]$,
 → Do not decrease k .
 Compare $T[i]$ and $P[k]$: 'C' = 'C'.
 → Increment k by 1 ($k = 5$).
 ($match[8] = k = 5$)
 $k = 5$ (the length of P).
 → Print "Pattern occurs with shift" $i - 5 + 1$.
 → Decrease k : $k = LPS[k - 1] = 0$.

Increment i by 1 ($i = 9$).

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Text	A	B	B	C	B	A	B	A	C	B	A	B	A
Pattern	B	A	B	A	C								
LPS[i]	0	0	1	2	0								
match[i]	0	1	1	0	1	2	3	4	5				

- Step 12: Consider $i = 9$ ($k = 0$).

Since $k = 0$,

→ Do not decrease k .

Compare $T[i]$ and $P[k]$: 'B' = 'B'.

→ Increment k by 1 ($k = 1$).

($\text{match}[9] = k = 1$)

$k \neq 5$ (the length of P).

→ Do not decrease k .

Increment i by 1 ($i = 10$).

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Text	A	B	B	C	B	A	B	A	C	B	A	B	A
Pattern	B	A	B	A	C								
LPS[i]	0	0	1	2	0								
match[i]	0	1	1	0	1	2	3	4	5	1			

- Step 13: Consider $i = 10$ ($k = 1$).

Since $k > 0$ and $T[i] = P[k]$,

→ Do not decrease k .

Compare $T[i]$ and $P[k]$: 'A' = 'A'.

→ Increment k by 1 ($k = 2$).

($\text{match}[10] = k = 2$)

$k \neq 5$ (the length of P).

→ Do not decrease k .

Increment i by 1 ($i = 11$).

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Text	A	B	B	C	B	A	B	A	C	B	A	B	A
Pattern	B	A	B	A	C								
LPS[i]	0	0	1	2	0								
match[i]	0	1	1	0	1	2	3	4	5	1	2		

- Step 14: Consider $i = 11$ ($k = 2$).
 Since $k > 0$ and $T[i] = P[k]$,
 → Do not decrease k .
 Compare $T[i]$ and $P[k]$: 'B' = 'B'.
 → Increment k by 1 ($k = 3$).
 ($match[11] = k = 3$)
 $k \neq 5$ (the length of P).
 → Do not decrease k .
 Increment i by 1 ($i = 12$).

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Text	A	B	B	C	B	A	B	A	C	B	A	B	A
Pattern	B	A	B	A	C								
LPS[i]	0	0	1	2	0								
match[i]	0	1	1	0	1	2	3	4	5	1	2	3	

- Step 15: Consider $i = 12$ ($k = 3$).
 Since $k > 0$ and $T[i] = P[k]$,
 → Do not decrease k .
 Compare $T[i]$ and $P[k]$: 'A' = 'A'.
 → Increment k by 1 ($k = 4$).
 ($match[12] = k = 4$)
 $k \neq 5$ (the length of P).
 → Do not decrease k .
 Increment i by 1 ($i = 13$).
 → End of the loop.

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Text	A	B	B	C	B	A	B	A	C	B	A	B	A
Pattern	B	A	B	A	C								
LPS[i]	0	0	1	2	0								
match[i]	0	1	1	0	1	2	3	4	5	1	2	3	4

3.4. Complexity

3.4.1. Time complexity

- Processing phase (building the LPS array):
 - By examining the pseudocode, one might mistakenly assume that the time complexity of the build LPS array function is $O(m^2)$, since there is a while loop nested inside a for loop. However, using the aggregate method of amortized analysis, we can show that the actual time complexity of building the LPS array is only $O(m)$.
 - More specifically, as stated in the introduction, $LPS[j] \leq j$ for every $0 \leq j < m$. Therefore, each time the while loop executes, the assignment $k = LPS[k - 1]$ strictly decreases the value of k .
 - On the other hand, k is always ≥ 0 , so the total number of executions of the while loop is at most the total amount by which k has increased in previous iterations of the for loop.
 - It is clear that the only operation that increases k is $k = k + 1$, and this increment occurs at most once per iteration of the for loop (when $P[k] = P[j]$). Since the for loop runs from 1 to $m - 1$, the total increase of k throughout the entire loop is strictly less than m .
 - In conclusion, although the while loop is nested inside the for loop, the total number of executions of the while loop is bounded above by m . Hence, the combined total number of iterations of both loops is at most $m + m$, and the overall time complexity is $O(m)$.
- Search phase:
 - Similarly to the construction of the LPS array, the number of times the while loop executes during the matching process is bounded above by the maximum total increase of k throughout the for loop.

- Since the for loop runs from 0 to $n - 1$, each iteration contains at most one operation $k = k + 1$, which increases k by at most one. Therefore, the total number of executions is bounded above by $n + n$, and the time complexity is $O(n)$.
- Therefore, the total time complexity of the KMP algorithm is $O(n + m) \approx O(n)$ (because $|T| \geq |P|$).

3.4.2. Space complexity

- Build LPS array: This preprocessing phase requires an auxiliary array of length m (equal to the length of the pattern P) to store the length of the longest proper prefix that is also a suffix of P at each position j . Therefore, the space complexity is $O(m)$.
- Search phase: This phase does not require any additional auxiliary array; it only uses a few scalar variables to store intermediate values. Hence, the space complexity is constant and does not depend on the input size, i.e., $O(1)$.
- Overall, the total space complexity of the algorithm is $O(m) + O(1) \approx O(m)$.

3.5. Discussion

- In the best-case scenario, the first character of the pattern P ($P[0]$) does not appear in the text T . In this case, k always remains 0, and the condition of the while loop ($k > 0$ and $T[i] \neq P[k]$) is always false, so the while loop is never executed. The algorithm then runs exactly n iterations. Moreover, since the construction of the LPS array depends only on the pattern, KMP performs very well on streaming data.
- On the other hand, for highly repetitive strings, the while loop has more opportunities to execute. In such cases, the actual number of character comparisons approaches $2n$.
- This is precisely the strength of the KMP algorithm compared to the naive brute-force approach. In all cases, the time complexity of the algorithm remains stable at $O(n)$.
- However, in practical natural-language text search applications (such as text editors), KMP is not always as efficient as the Boyer–Moore algorithm.
- In conclusion, although KMP is not the fastest algorithm in every situation, its greatest advantage lies in its excellent stability in the number of iterations across all datasets.

4. The Boyer-Moore algorithm

4.1. Introduction & Application

4.1.1. Introduction [5]

- The main idea of the Boyer-Moore algorithm is to preprocess the information given from the original string (called *Text*, denoted by *T*) and the string to be searched for (called *Pattern*, denoted by *P*) so the unnecessary comparison steps can be skipped, thereby speeding up the program's processing.
- To achieve this, Boyer-Moore contains three clever ideas not contained in the naive algorithm: the right-to-left scan, the Bad character heuristic, the Good suffix heuristic. Together, these ideas lead to a method that typically examines fewer than $|T| + |P|$ characters (an expected sublinear-time method) and that (with a certain extension) runs in linear worst-case time.
- Comparing characters from right-to-left isn't any better than checking from left-to-right. However, with two additional ideas, shifts of more than one position often occur, and in typical situations large shifts are common.
- **Bad Character Heuristic:** When a mismatch occurs during the right-to-left comparison of the pattern $|P|$ and the text $|T|$, the algorithm calculates a safe shift distance based on the right-most occurrence of the mismatched text character within $|P|$:
 - Let $\text{badChar}[x]$ denote the right-most occurrence of character x in P . $\text{badChar}[x]$ is defined to be -1 (negative one) if x does not occur in P .
 - If a mismatch occurs at position i of the pattern against character $T[k]$ in the text, P is shifted to the right by $\max(1, i - \text{badChar}[T[k]])$ places.
- Modern Boyer-Moore implementations replace the original Basic Good Suffix heuristic with the Strong variant. Despite sharing identical complexity and preprocessing time, the Strong version resolves a fatal algorithmic blind spot. Consequently, this report exclusively covers the principles and complexity of the Strong Good Suffix heuristic (hereafter referred to simply as the "Good Suffix Heuristic").
- **Good Suffix Heuristic:** When a mismatch occurs after successfully matching a suffix t of the pattern P against the text T , the algorithm shifts P to the right to align T with another occurrence of t found earlier within P . Crucially, the "strong" condition dictates that the character immediately preceding this new occurrence in P must be different from the character that just caused the mismatch.
 - Let $\text{goodSuf}[i]$ denote the optimal shift distance when a mismatch occurs at index i of pattern P . It is defined as the distance to shift P to the right so that the successfully matched suffix $P[i+1..|P|-1]$ aligns with its right-most occurrence in P , strictly provided that the character immediately preceding

this occurrence is not equal to $P[i]$. If no such occurrence exists, $\text{goodSuf}[i]$ is defined as the shift distance to align the longest proper prefix of P that perfectly matches a suffix of $P[i+1..m-1]$. If neither condition is met, $\text{goodSuf}[i]$ is defined to be $|P|$.

- To compute these values efficiently, an auxiliary array is utilized. Let $\text{bPos}[i]$ (border position) denote the starting index of the widest border of the suffix $P[i..|P|-1]$. This array helps prevent redundant character comparisons during the preprocessing phase.
- If a mismatch occurs at position i (meaning the suffix t from index $i+1$ to the end of P matched perfectly), the pattern is shifted according to three descending priorities:
 1. Full Match: Find the right-most occurrence of t in P where the preceding character is not equal to $P[i]$. Shift P to align this occurrence with t in T .
 2. Partial Prefix Match: If no such full occurrence exists, find the longest proper prefix of P that perfectly matches a suffix of t . Shift P to align this prefix.
 3. No Match: If neither exists, shift the entire pattern P right by its full length $|P|$ (shifting completely past t).
- The Boyer-Moore algorithm combines these two heuristics by choosing the maximum shift proposed by either of them at each step, as both guarantee that no potential matches are missed.

4.1.2. Application[6]

- The Boyer-Moore algorithm is one of the most efficient for string matching, and is also considered the foundation for developing better string matching algorithms.
- The Boyer-Moore is widely applied in:
 - Text editor: Used for fast find-and-replace operations, where quick pattern matching improves responsiveness, especially in large files.
 - Search engine: Skips irrelevant text parts to enable effective keyword searches across big document volumes.
 - Bioinformatics: Used in DNA and RNA sequence matching, where quick and memory-efficient pattern searches are necessary for big biological datasets
 - Compilers: Used in lexical analysis and syntax tree processing to identify patterns in source code efficiently.

4.2. Pseudocode

```
1  let badChar[0..256] be a new array //256 for ASCII
2  for i: 0 → 255
3      badChar[i] = -1
4  for i: 0 → |P|-1
5      badChar[P[i]] = i

6  //Pre-processing good suffix heuristic
7  let goodSuf[0..|P|] and bPos[0..|P|] be a new array
8  for i: 0 → |P|-1
9      goodSuf[i] = 0

10 //Case 1: Suffix matching elsewhere
11 i = |P|
12 j = |P| + 1

13 bPos[i] = j
14 while i > 0:
15     while j ≤ |P| and P[i - 1] ≠ P[j - 1]
16         if goodSuf[j] == 0
17             goodSuf[j] = j - i
18         j = bPos[j]
19     i = i - 1
20     j = j - 1
21     bPos[i] = j

22 //Case 2: Prefix matching suffix
23 j = bPos[0]
24 for i: 0 → |P|
25     if goodSuf[i] == 0
26         goodSuf[i] = j
27     if i == j
28         j = bPos[j]

29 //Search phase
```

```

30 s = 0 //s is the shift of the pattern
31 while s ≤ |T| - |P|
32     j = |P| - 1 //Start matching from right to left
33     while j ≥ 0 and P[j] == T[s + j]
34         j = j - 1
35     if j < 0
36         print "Pattern occurs with shift" s
37         s = s + goodSuf[0]
38     else
39         //bad character heuristic shift
40         s_bc = j - badChar[T[s + j]]
41         //good suffix heuristic shift
42         s_gs = goodSuf[j + 1]
43         s = s + max(s_bc, s_gs)

```

4.3. Step-by-step trace

- Given the text string $T = \text{"BABABACA"}$ and the pattern string $P = \text{"BACA"}$.

4.3.1. Pre-processing bad character heuristic

- Step 1: Initialize the badChar array with 256 elements.
- Step 2: Set all values of badChar = -1

Index	'A'	'B'	'C'
badChar[i]	-1	-1	-1

- Step 3: We find the rightmost occurrence of each character in pattern P. Initialize $i = 0$ as the loop index. .
- Step 4: Consider $i = 0, P[i] = B$, update badChar[P[i]] = 0.

Index	'A'	'B'	'C'
badChar[P[i]]	-1	0	-1

- Step 5: Consider $i = 1$, $P[i] = A$, update $\text{badChar}[P[i]] = 1$

Index	'A'	'B'	'C'
$\text{badChar}[P[i]]$	1	0	-1

- Step 6: Consider $i = 2$, $P[i] = C$, update $\text{badChar}[P[i]] = 2$

Index	'A'	'B'	'C'
$\text{badChar}[P[i]]$	1	0	2

- Step 7: Consider $i = 3$, $P[i] = A$, update $\text{badChar}[P[i]] = 3$

Index	'A'	'B'	'C'
$\text{badChar}[P[i]]$	3	0	2

4.3.2. Pre-processing good suffix heuristic

- Step 1: Initialize the goodSuf array with 5 elements (as $|P| + 1$) and the bPos array with 5 elements (as $|P| + 1$).
- Step 2: Initialize $i = 4$ and $j = 5$.
- Step 3: Set $\text{bPos}[i] = j$ ($\text{bPos}[4] = 5$).
- Step 4: Start processing case 1 (Suffix matching elsewhere). While loop with condition $i > 0$.

index	0	1	2	3	4
$\text{goodSuf}[i]$	0	0	0	0	0
$\text{bPos}[i]$	0	0	0	0	5

- Step 5: Consider $i = 4$.

Since $j = 5$, the condition $j \leq 4$ is wrong. We skip the while loop inside
Decrement i by 1, decrement j by 1.

→ $i = 3$, $j = 4$

Set $\text{bPos}[i] = j$ ($\text{bPos}[3] = 4$)

index	0	1	2	3	4
$\text{goodSuf}[i]$	0	0	0	0	0
$\text{bPos}[i]$	0	0	0	4	5

- Step 6: Consider $i = 3$

Since $j \leq 4$ and $P[i-1] ('C') \neq P[j-1] ('A')$. We run the while loop inside.

Since $\text{goodSuf}[j] = 0$

$\rightarrow \text{goodSuf}[j] = j - i$ ($\text{goodSuf}[4] = 1$)

Set $j = \text{bPos}[j]$ ($j = 5$)

Since $j = 5$, we break the while loop inside.

Decrement i by 1, decrement j by 1

$\rightarrow i = 2, j = 4$

Set $\text{bPos}[i] = j$ ($\text{bPos}[2] = 4$)

index	0	1	2	3	4
$\text{goodSuf}[i]$	0	0	0	0	1
$\text{bPos}[i]$	0	0	4	4	5

- Step 7: Consider $i = 2$

Since $P[i-1] ('A') = P[j-1] ('A')$, the condition $P[i-1] \neq P[j-1]$ is wrong. We skip the while loop inside.

Decrement i by 1, decrement j by 1

$\rightarrow i = 1, j = 3$

Set $\text{bPos}[i] = j$ ($\text{bPos}[1] = 3$)

index	0	1	2	3	4
$\text{goodSuf}[i]$	0	0	0	0	1
$\text{bPos}[i]$	0	3	4	4	5

- Step 8: Consider $i = 1$

Since $j \leq 4$ and $P[i-1] ('B') \neq P[j-1] ('C')$. We run the while loop inside.

Since $\text{goodSuf}[j] = 0$

$\rightarrow \text{Set } \text{goodSuf}[j] = j - i$ ($\text{goodSuf}[3] = 2$)

$j = \text{bPos}[j]$ ($j = 4$)

Since $j \leq 4$ and $P[i-1] ('B') \neq P[j-1] ('A')$.

Since $\text{goodSuf}[j] \neq 0$

$\rightarrow$ Do not update $\text{goodSuf}[j]$

$j = \text{bPos}[j]$ ($j = 5$)

Since $j = 5$, the condition $j \leq 4$ is wrong. We break the while loop inside.

Decrement i by 1, decrement j by 1

→ $i = 0, j = 4$

Set $bPos[i] = j$ ($bPos[0] = 4$)

index	0	1	2	3	4
goodSuf[i]	0	0	0	2	1
bPos[i]	4	3	4	4	5

- Step 9: Consider $i = 0$, the condition $i > 0$ is wrong. We stop the while loop.
- Step 10: Start processing case 2 (Suffix matching suffix), we continue to fill 0's positions in the goodSuf array.

Set $j = bPos[0]$ ($j = 4$)

- Step 11: Initialize $i = 0$ as the loop index.
- Step 12: Consider $i = 0$.

Since $goodSuf[i] = 0$ ($goodSuf[0] = 0$)

→ $goodSuf[i] = j$ ($goodSuf[0] = 4$)

Since $i \neq j$, we do not update j

index	0	1	2	3	4
goodSuf[i]	4	0	0	2	1
bPos[i]	4	3	4	4	5

- Step 13: Consider $i = 1$

Since $goodSuf[i] = 0$ ($goodSuf[1] = 0$)

→ $goodSuf[i] = j$ ($goodSuf[1] = 4$)

Since $i \neq j$, we do not update j

index	0	1	2	3	4
goodSuf[i]	4	4	0	2	1
bPos[i]	4	3	4	4	5

- Step 14: Consider $i = 2$

Since $goodSuf[i] = 0$ ($goodSuf[2] = 0$)

→ $goodSuf[i] = j$ ($goodSuf[2] = 4$)

Since $i \neq j$, we do not update j

index	0	1	2	3	4
-------	---	---	---	---	---

goodSuf[i]	4	4	4	2	1
bPos[i]	4	3	4	4	5

- Step 15: Consider $i = 3$

Since $\text{goodSuf}[i] = 2$ ($\text{goodSuf}[3] = 2$)

→ we do not update $\text{goodSuf}[i]$

Since $i \neq j$, we do not update j

index	0	1	2	3	4
goodSuf[i]	4	4	4	2	1
bPos[i]	4	3	4	4	5

- Step 16: Consider $i = 4$

Since $\text{goodSuf}[i] = 1$ ($\text{goodSuf}[4] = 1$)

→ we do not update $\text{goodSuf}[i]$

Since $i = j$

→ $j = \text{bPos}[j]$ ($j = 5$)

index	0	1	2	3	4
goodSuf[i]	4	4	4	2	1
bPos[i]	4	3	4	4	5

4.3.3. Matching phase

- Step 1: Initialize $s = 0$, s is the shift of P . Run while loop with condition $s \leq |T| - |P|$.

$s = 0$								
index	0	1	2	3	4	5	6	7
Text	B	A	B	A	B	A	C	A
Pattern	B	A	C	A				

- Step 2: Consider $s = 0$.
Set $j = |P| - 1$ ($j = 3$)
Compare character in T and P

Consider $j = 3$.

Since $P[j] ('A') = T[s + j] ('A')$

→ Decrement j by 1.

$s = 0$								
index	0	1	2	3	4	5	6	7
Text	B	A	B	A	B	A	C	A
Pattern	B	A	C	A				

Consider $j = 2$

Since $P[j] ('C') \neq T[s+j] ('B')$

→ Wrong!

$s = 0$								
index	0	1	2	3	4	5	6	7
Text	B	A	B	A	B	A	C	A
Pattern	B	A	C	A				

Calculate shifts:

Bad Character Shift: $j - \text{badChar}[T[s + j]] = 2 - \text{badChar}['B'] = 2 - 0 = 2$

Good Suffix Shift: $\text{goodSuf}[j + 1] = \text{goodSuf}[3] = 2$

$s = s + \max(2, 2) \rightarrow s = 2$

- Step 3: Consider $s = 2$

Set $j = |P| - 1$ ($j = 3$)

Compare character in T and P

Consider $j = 3$

Since $P[j] ('A') = T[s + j] ('A')$

→ Decrement j by 1

$s = 2$								
index	0	1	2	3	4	5	6	7
Text	B	A	B	A	B	A	C	A
Pattern	B	A	C	A				

Consider $j = 2$

Since $P[j] ('C') \neq T[s + j] ('B')$

→ Wrong!

$s = 2$								
index	0	1	2	3	4	5	6	7
Text	B	A	B	A	B	A	C	A
Pattern	B	A	C	A				

Calculate shifts

Bad Character Shifts: $j - \text{badChar}[T[s + j]] = 2 - \text{badChar}['B'] = 2$

- 0 = 2

Good Suffix Shifts: $\text{goodSuf}[j + 1] = \text{goodSuf}[3] = 2$

$s = s + \max(2, 2) \rightarrow s = 4$

- Step 4: Consider $s = 4$

Set $j = |P| - 1$ ($j = 3$)

Compare character in T and P

Consider $j = 3$

Since $P[j] ('A') = T[s + j] ('A')$

→ Decrement j by 1

$s = 4$								
index	0	1	2	3	4	5	6	7
Text	B	A	B	A	B	A	C	A
Pattern	B	A	C	A				

Consider $j = 2$

Since $P[j] ('C') = T[s + j] ('C')$

→ Decrement j by 1

$s = 4$								
index	0	1	2	3	4	5	6	7
Text	B	A	B	A	B	A	C	A
Pattern	B	A	C	A				

Consider $j = 1$

Since $P[j] ('A') = T[s + j] ('A')$

→ Decrement j by 1

$s = 4$								
index	0	1	2	3	4	5	6	7
Text	B	A	B	A	B	A	C	A
Pattern	B	A	C	A				

Consider $j = 0$

Since $P[j] ('B') = T[s + j] ('B')$

→ Decrement j by 1

$s = 4$								
index	0	1	2	3	4	5	6	7
Text	B	A	B	A	B	A	C	A
Pattern	B	A	C	A				

Since $j = -1 \rightarrow$ **Found the P in T!**

→ $s = s + \text{goodSuf}[0] = 4 + 4 = 8$. To find subsequent occurrences, shift by the whole pattern boundary.

4.4. Complexity

4.4.1. Time complexity [5], [7]

- Processing Bad Character Heuristic phase:
 - The algorithm uses a for loop to initialize an array of size Σ (which Σ is alphabet size) with a default value of -1. This operation takes $O(\Sigma)$ time. Next, a for loop iterates through the entire pattern P of length $|P|$ to update the character positions. This operation takes $O(|P|)$. The overall time complexity of these two processes is $O(|P| + \Sigma)$.
- Processing Good Suffix Heuristic phase:
 - This phase creates the goodSuf array and an auxiliary bPos array (as known as border position).

- Look at the pseudocode, we see a nested while loop. The natural reflex is to assume the complexity will be $O(m^2)$. However, it is actually only $O(m)$. The algorithm relies on computing "borders" (the longest proper prefix that is also a suffix).
- Inside the nested while loop (in Case 1), the value of the variable j is continuously decremented through the assign $j = \text{bPos}[j]$. The total number of times in the inner loop execute cannot exceed the total number of possible borders of the pattern. Amortized Analysis proves that the maximum number of computational steps is upper-bounded by a constant multiplied by $|P|$. Therefore, it runs in linear time: $O(|P|)$.
- Searching phase
 - The best case: $O(\lfloor |T| / |P| \rfloor)$
 - This happens when the last character of the pattern P consistently mismatches with the text T , and that mismatched character does not exist anywhere within the pattern P .
 - The algorithm applies the Bad Character rule and immediately shifts the entire pattern by $|P|$ positions. For a text of length $|T|$, the algorithm only needs to perform approximately $\lfloor |T| / |P| \rfloor$ comparisons. This is the only string-searching algorithm capable of achieving sub-linear speed; the longer the pattern, the faster the search!
 - The worst case: $O(|T|)$.
 - In original version of Boyer-Moore, which is using Bad Character Heuristic and Good Suffix Heuristic (weak rule), the worst scenario this version faces is time complexity could reach $O(|T| * |P|)$ when the problem requires all occurrences of the pattern in text. This happens with periodic or highly repetitive patterns. For example, $T = \text{"AAAAAA..."}$, $P = \text{"AAAA..."}$.
 - However, by using the Strong Good Suffix Heuristic the time complexity can be reduced to $O(|T|)$. This was first proved by Knuth, Morris, and Pratt, and an alternate proof was given by Guibas and Odlyzko. Both of these proofs were quite difficult and established worst-case time bounds no better than $5|T|$ comparisons. Later, Richard Cole gave a much simpler proof establishing a bound of $4|T|$ comparisons and also gave a difficult proof establishing a tight bound of $3|T|$ comparisons. More precisely, an upper bound of $3|T| - 3(|T| - |P| + 1) / (|P| + 2)$ comparisons are shown:

$$C_{\max} \leq 3n - 3 * (n - m + 1) / (m + 2).$$

4.4.2. Space complexity

- Processing Bad Character Heuristic phase
 - The only memory allocation required for this part is the badChar array, which has a size equal to the total number of characters in the alphabet Σ (typically a 256-element integer array). The overall space complexity is $O(\Sigma)$.
- Processing Good Suffix Heuristic phase
 - We need to allocate two arrays: the goodSuf array and the bPos array. Both have a size of $|P| + 1$. Storing these arrays requires memory space directly proportional to the length of the pattern: $O(|P|)$.
- Searching phase
 - This is where the algorithm demonstrates its true value on long texts. Memory allocation in this step only costs $O(1)$ for the pointers (s, j) , so our focus is entirely on time efficiency.
 - During the search process, the algorithm only reads data from the previously created badChar and goodSuf arrays. Therefore, the overall space complexity of the entire Boyer-Moore algorithm remains $O(m + \Sigma)$.

4.5. Discussion [6]

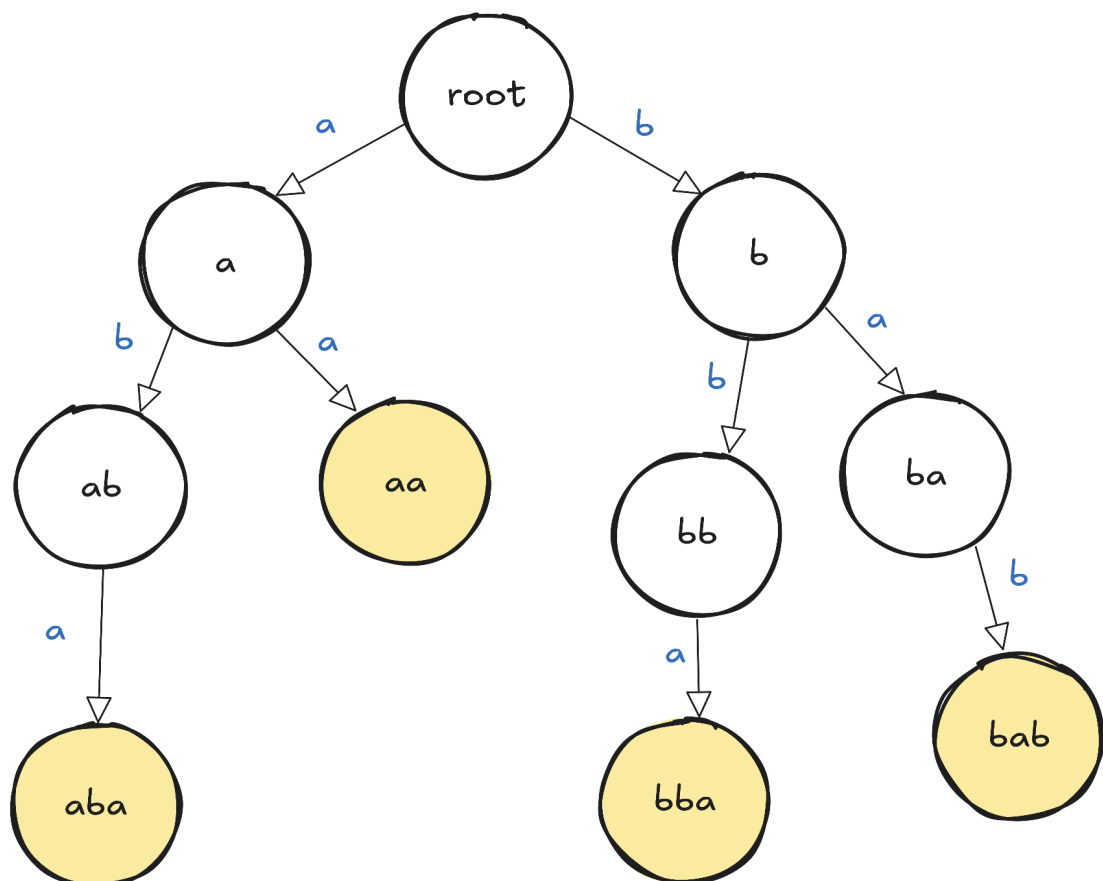
- The Boyer-Moore algorithm works strongly in natural languages. Because natural language naturally avoids the algorithm's kryptonite, Boyer-Moore almost never degrades to its worst-case bounds in everyday text searching. It frequently outperforms other algorithms, such as KMP or Rabin-Karp, in real-world scenarios.
- Because of the large alphabet and the lack of periodicity in human languages, the algorithm operates almost exclusively in its best-case scenario. Instead of inspecting every single character in a document, it bounds across the text, checking only a fraction of the total characters. It actually runs faster as the search pattern gets longer.
- However, The Boyer-Moore is still not optimal in all cases. In binary text or DNA text, which has high repetitive and periodicity, this algorithm is not always as efficient as the KMP algorithm.
- Today, numerous variations of this algorithm exist, designed to optimize its efficiency and tackle highly specialized scenarios.

5. Aho-Corasick

5.1. Introduction & Application

5.1.1. Introduction

- String matching algorithms such as Rabin–Karp and KMP are highly efficient for matching a single pattern P against a text T , achieving an average time complexity of $O(n)$. However, when the number of patterns increases, the algorithm must repeatedly scan the text T (which is usually much longer than each pattern). To address this issue, the Aho–Corasick algorithm was developed. It efficiently matches multiple patterns simultaneously by combining the Trie data structure with ideas from the KMP algorithm.
- A Trie is a tree-like data structure built from a set of strings, where each node represents a prefix of one or more patterns in the set. Trie operations typically include insertion, deletion, and search [7]. Below is a Trie constructed from the strings “aba”, “aa”, “bba”, and “bab”:



- The Aho–Corasick algorithm leverages the efficient data organization of the Trie to improve upon the LPS-based approach used in the KMP algorithm. Instead of storing the longest proper prefix that is also a suffix for a single pattern, it uses a structure called a failure link to indicate the next position to transition to within the set of patterns (the Trie) when a mismatch occurs [8].

- In addition, at each node of the Trie, the algorithm stores the patterns that end at that node and connects them to other matched patterns in different branches of the Trie using dictionary links, ensuring that no matches are missed [8].

5.1.2. Application

- Similar to other string matching algorithms, Aho–Corasick is used for keyword searching in text editors, plagiarism detection, and related applications.
- Moreover, due to its ability to match multiple patterns simultaneously, Aho–Corasick is also widely applied in network security software for detecting malicious programs, as well as in content filtering systems for identifying prohibited words on social media platforms [9].

5.2. Pseudocode

```
1  //Build Trie from a set of Keywords
2  For each Keyword in set
3      curNode = Root
4      For each character C in Keyword:
5          If child(curNode, C) does not exist:
6              Create new node V
7              child(curNode, C) = V
8              curNode = child(curNode, C)
9      Add ID to FoundWordIDs(curNode)

10 //Build failure links and dictionary links
11 BuildAutomaton():
12     Initialize Queue Q
13     For each character C in Alphabet:
14         If child(Root, C) exists:
15             V = child(Root, C)
16             Fail(V) = Root
17             Next(Root, C) = V
18             Push V to Q
19         Else:
20             Next(Root, C) = Root
```

```

21   While Q is not empty:
22       U = Pop from Q
23       For each character C in Alphabet:
24           If child(U, C) exists:
25               V = child(U, C)
26               Fail(V) = Next(Fail(U), C)
27               // Dictionary link
28               If FoundWordIDs(Fail(V)) is not empty:
29                   DictLink(V) = Fail(V)
30               Else:
31                   DictLink(V) = DictLink(Fail(V))
32               Next(U, C) = V
33               Push V to Q
34       Else:
35           //using failure link
36           Next(U, C) = Next(Fail(U), C)

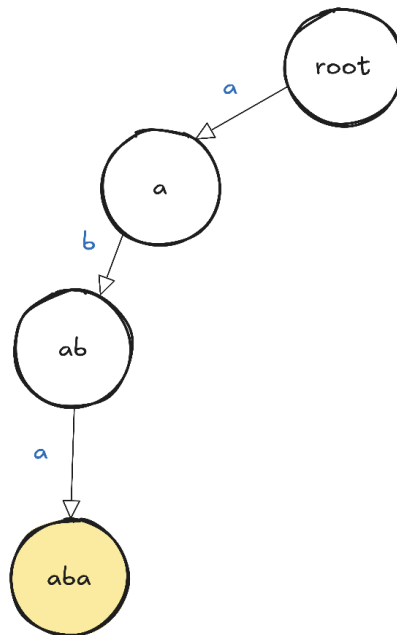
37 //Matching Keywords in Text
38 Search(Text):
39     curNode = Root
40     For i from 0 to length(Text) - 1:
41         C = Text[i]
42         curNode = Next(curNode, C)
43         //go to next state
44         Temp = curNode
45         While Temp != Root:
46             If FoundWordIDs(Temp) is not empty:
47                 For each ID in FoundWordIDs(Temp):
48                     Report found Keyword(ID) at position
49                         (i - |Keyword(ID)| + 1)
49             Temp = DictLink(Temp)

```

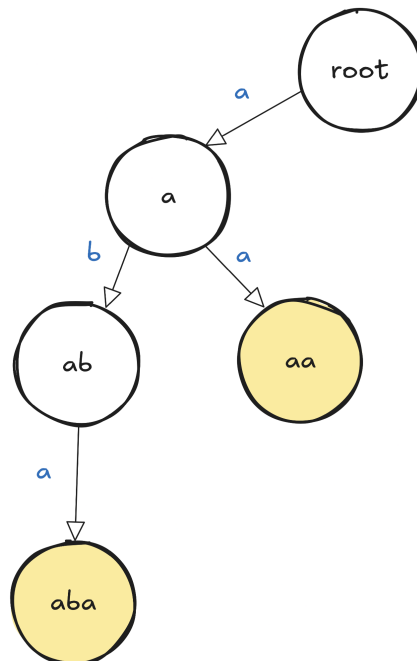
5.3. Step-by-step trace

- Given a set of Patterns = {"aba", "aa", "bba", "ba", "bab"} and the text string T = "abbaaababa".

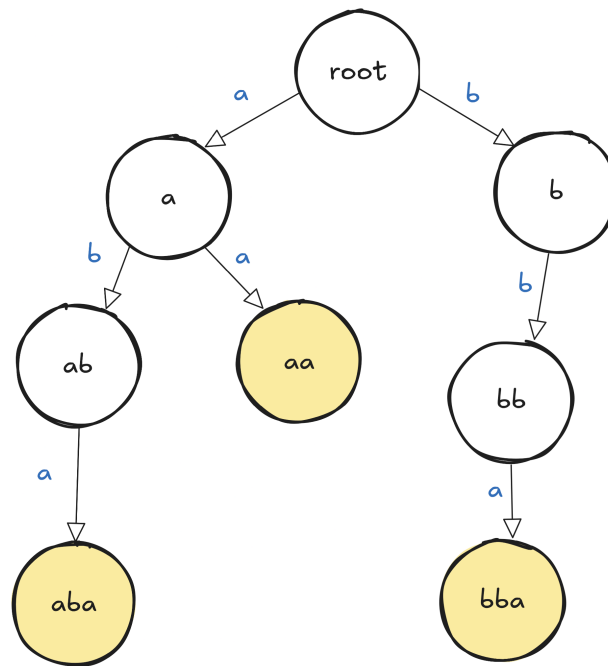
- Phase 1: Build the Trie.
- Step 1: Insert the string "aba".



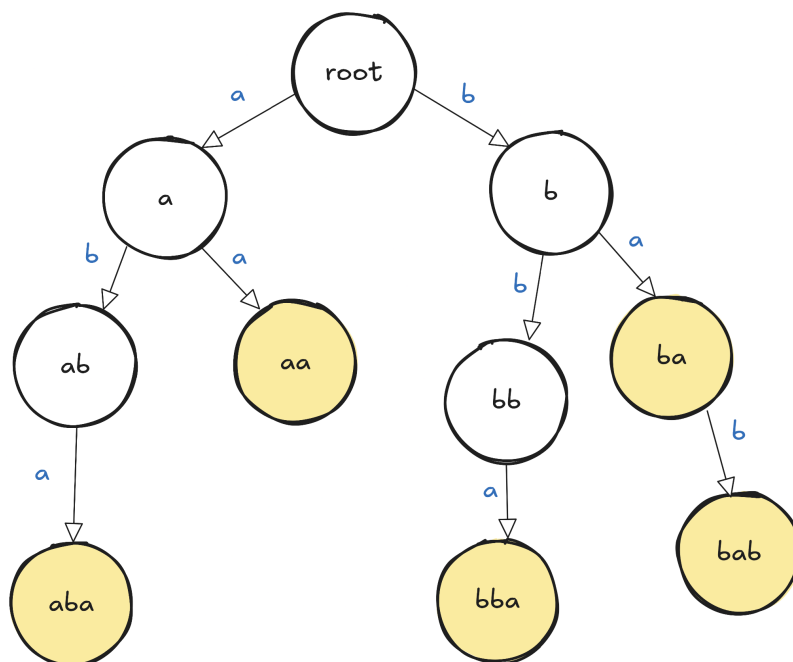
- Step 2: Insert the string "aa".



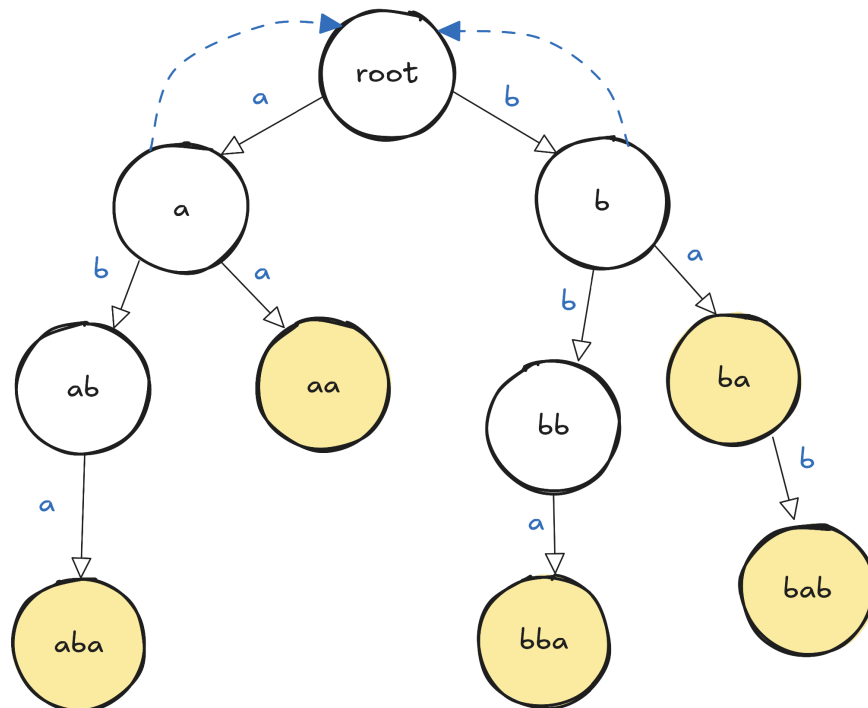
- Step 3: Insert the string "bba".



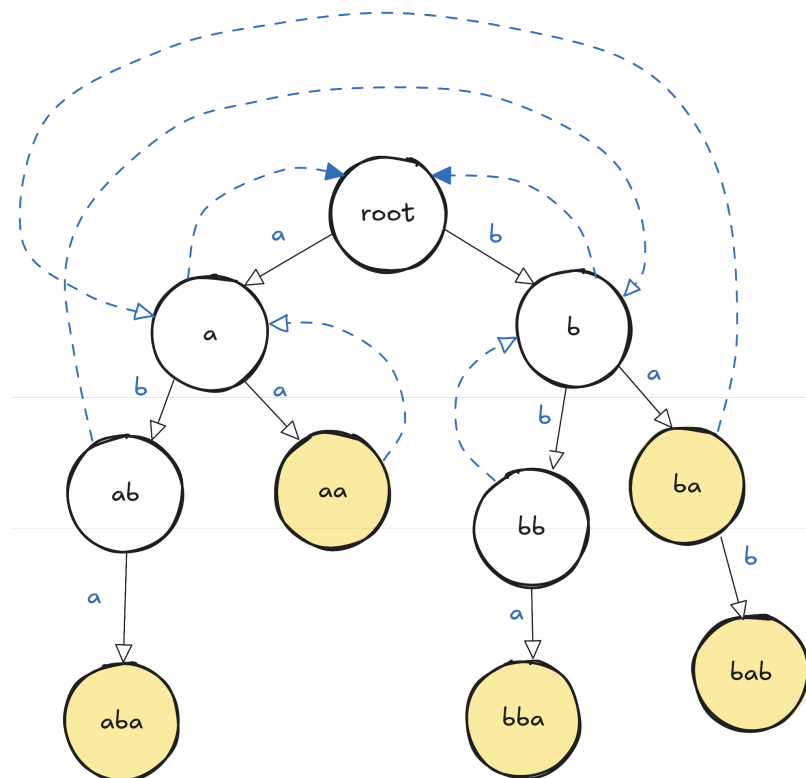
- Step 4: Insert the string “ba” and “bab”.



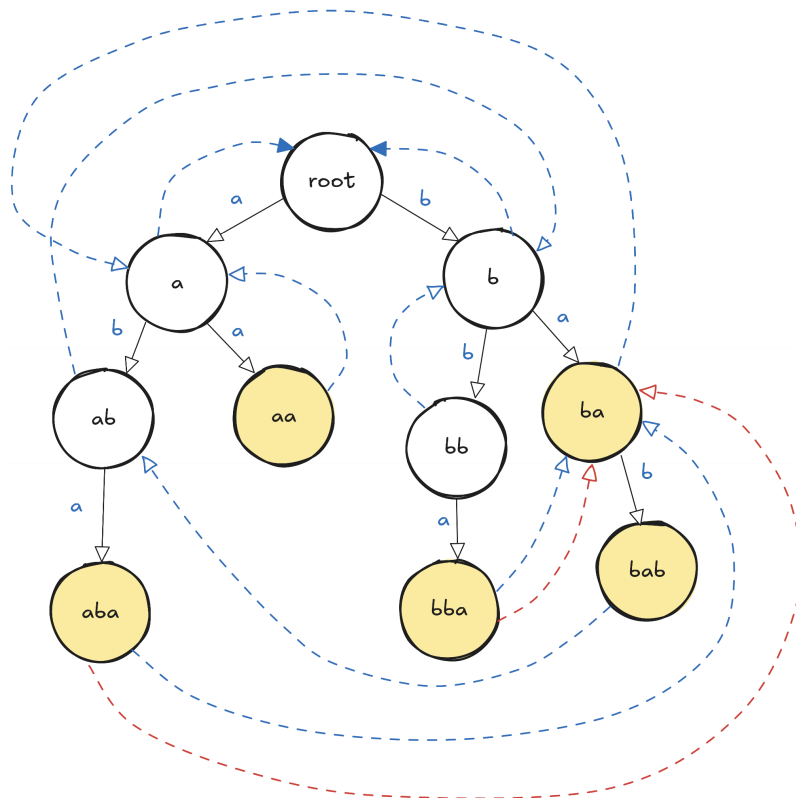
- Phase 2: Build failure links and dictionary links.
- Step 1: Set the failure links of all nodes that are direct children of the root to point back to the root.



- Step 2: Enqueue all direct children of the root.
- Step 3: Traverse the Trie using BFS (breadth-first search) to compute failure links and dictionary links (level 2).
 - The failure link of node "ab" points to node "b", node "aa" points to node "a", node "bb" points to node "b", and node "ba" points to node "a".
 - Nodes "aa" and "ba" correspond to complete patterns.

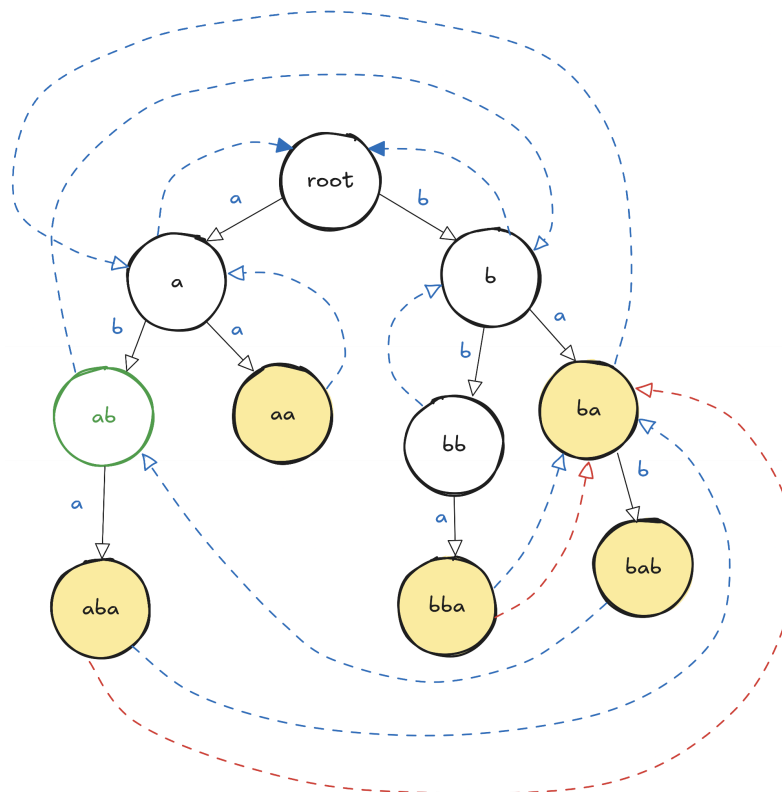
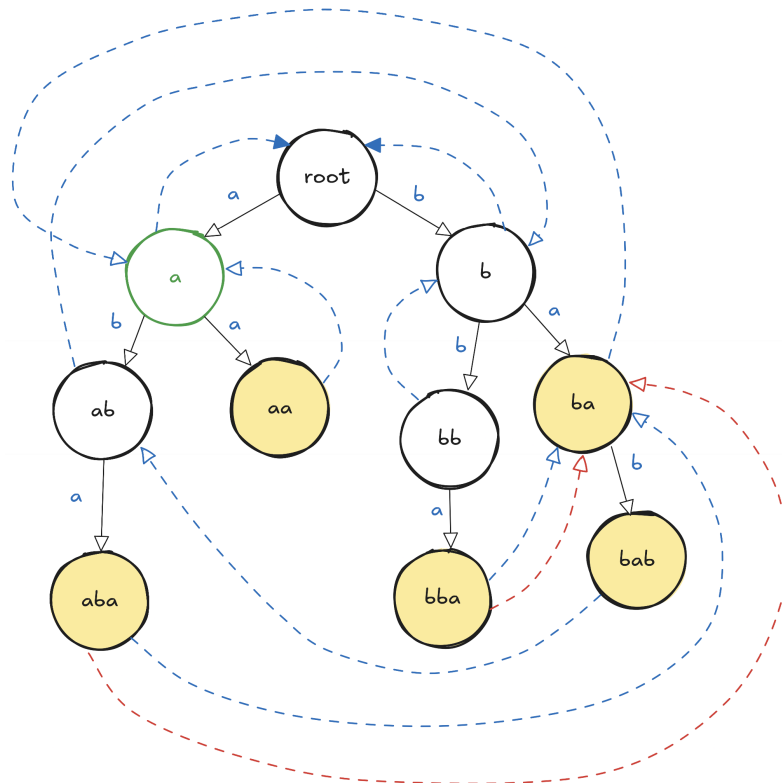


- Step 4: Continue traversing the Trie in BFS (breadth-first search) order to compute failure links and dictionary links (level 3).
 - The failure link of node "aba" points to node "ba", node "bba" points to node "ba", and node "bab" points to node "ab".
 - Nodes "aba" and "bba" correspond to complete patterns and have a suffix that is also a pattern ("ba"), so their dictionary links point to node "ba".
 - Node "bab" is also a complete pattern. Although reaching node "bab" requires passing through node "ba", none of its proper suffixes is a pattern, so there is no dictionary link from node "bab".

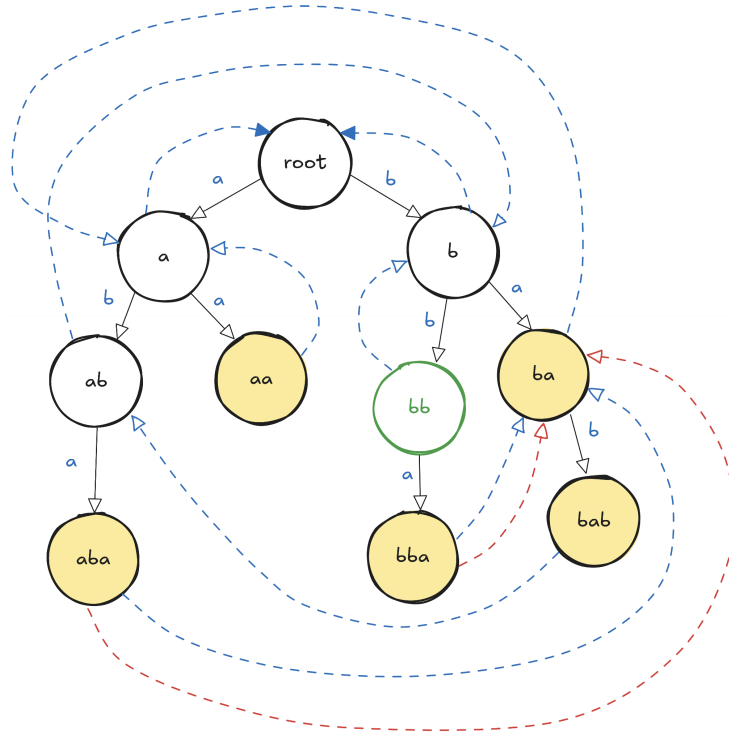


- Phase 3: Matching
- To make the matching process efficient, we construct a finite automaton based on the failure links, defined as follows:
- If we are at node u and the next character in the text is c , then:
 - $\text{next}(u, c) = v$ if there exists an edge labeled c from node u to node v in the Trie
 - $\text{next}(u, c) = \text{next}(\text{fail}(u), c)$ if no such edge exists and u is not the root
 - $\text{next}(u, c) = \text{root}$ if no such edge exists and u is the root
- Step 1: Start at the root and traverse each character of the text $T = \text{"abbbaababa"}$. The first character is 'a'. Based on the state transition definition, we move to node "a" (highlighted in green). Since no pattern ends at node "a", we continue with the next character in the text.

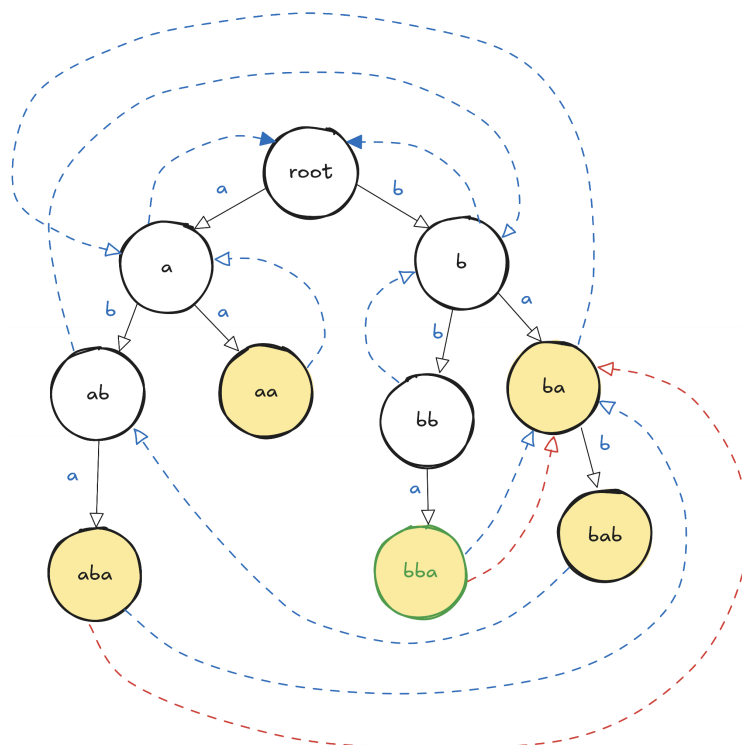
- Step 2: Continue with the second character in the text, which is 'b'. Based on the state transition definition, we move from node "a" to node "ab". No pattern ends at this node.



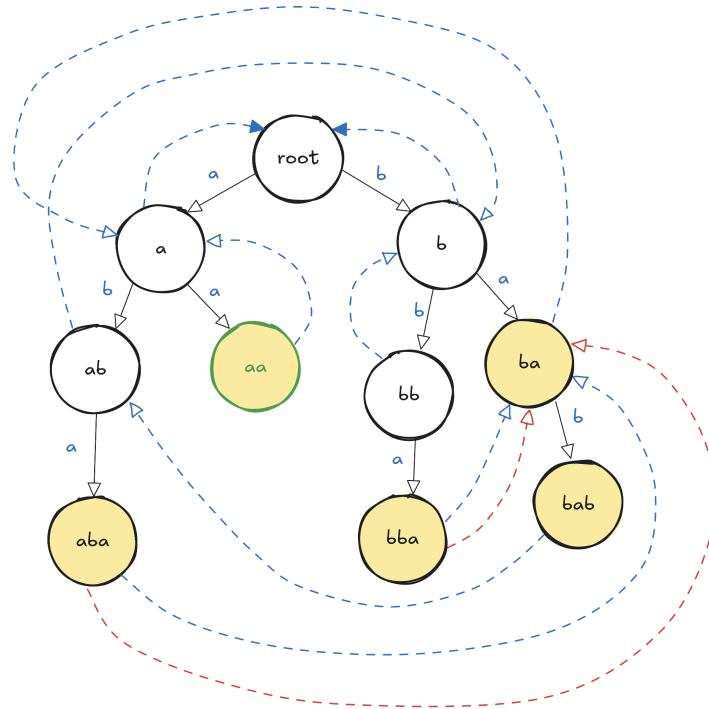
- Step 3: Continue with the third character in the text, which is 'b'. Based on the state transition definition, since there is no edge labeled 'b' from node "ab" and "ab" is not the root, we transition from node "ab" to node $\text{next}(\text{fail}(\text{"ab"}), 'b') = \text{"bb"}$. No pattern ends at this node.



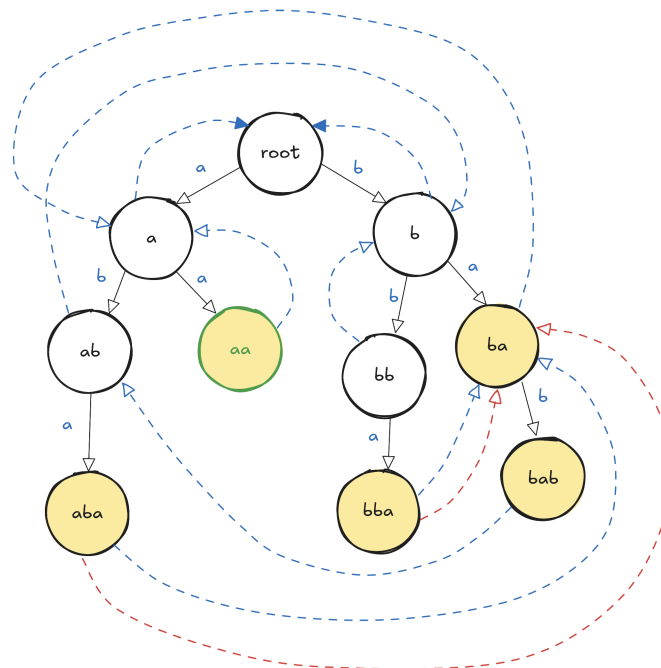
- Step 4: Continue with the fourth character in the text, which is 'a'. Based on the state transition definition, we move from node "bb" to node "bba". The node "bba" corresponds to one of the target patterns, so we record a match. Since node "bba" has a dictionary link pointing to node "ba", we also record the occurrence of the pattern "ba". Node "ba" has no further dictionary links, so no additional checks are needed.



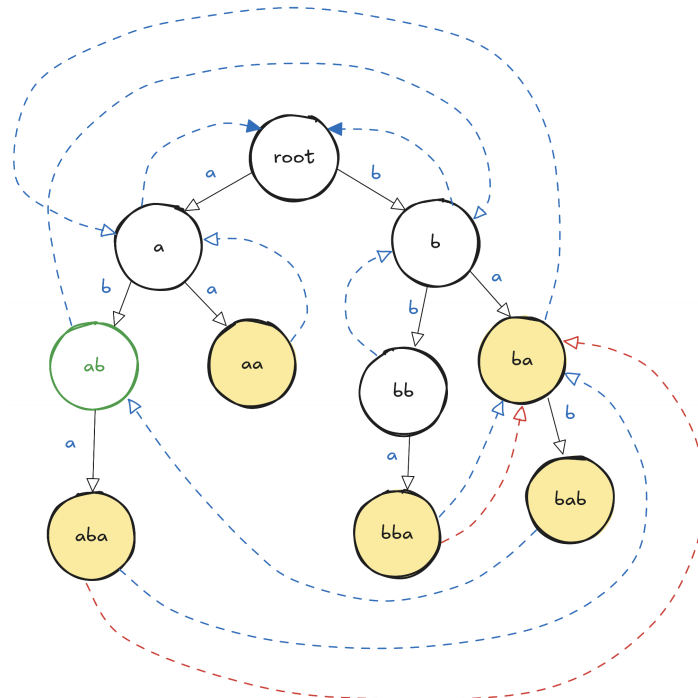
- Step 5: Continue with the fifth character in the text, which is 'a'. Based on the state transition definition, we move from node "bba" to node $\text{next}(\text{fail}(\text{"bba"}), 'a') = \text{"aa"}$. The node "aa" corresponds to one of the target patterns, so we record a match.



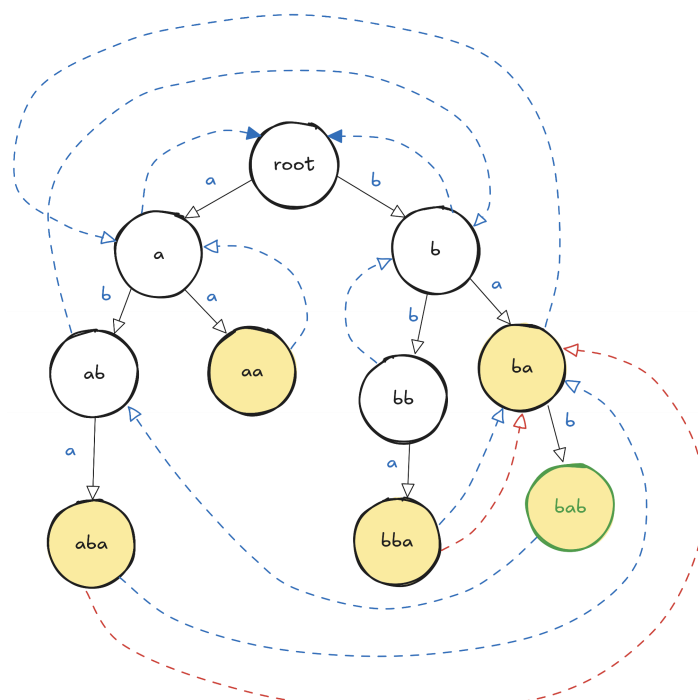
- Step 6: Continue with the sixth character in the text, which is 'a'. Based on the state transition definition, we move from node "aa" to node $\text{next}(\text{fail}(\text{"aa"}), 'a') = \text{"aa"}$. The node "aa" corresponds to one of the target patterns, so we record a match.



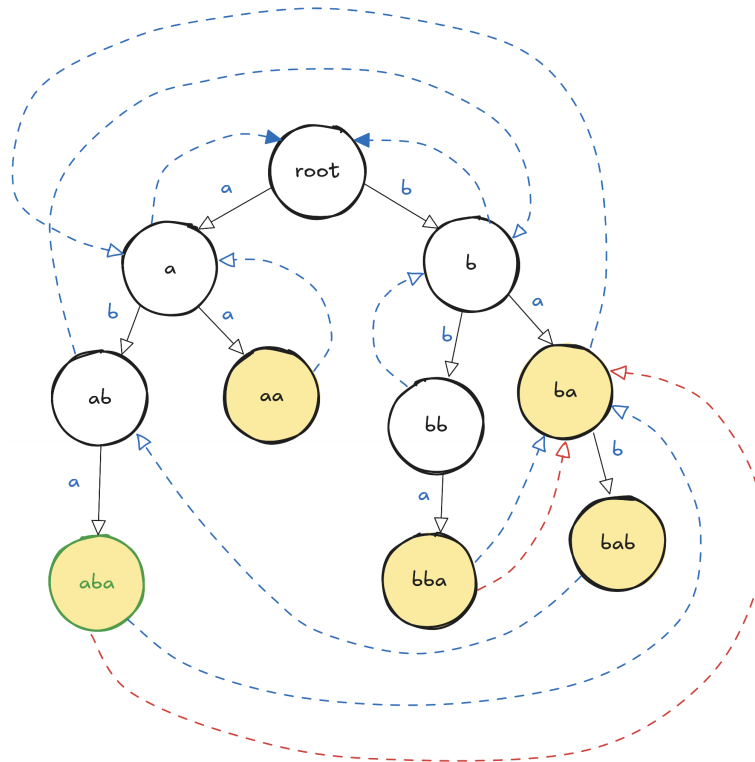
- Step 7: Continue with the seventh character in the text, which is 'b'. Based on the state transition definition, we move from node "aa" to node $\text{next}(\text{fail}(\text{"aa"}), 'b') = \text{"ab"}$. No pattern ends at this node.



- Step 8: Continue with the eighth character in the text, which is 'a'. Based on the state transition definition, we move from node "ab" to node "aba". The node "aba" corresponds to one of the target patterns, so we record a match. Since node "aba" has a dictionary link to node "ba", we also record the occurrence of the pattern "ba". Node "ba" has no further dictionary links, so we stop.
- Step 9: Continue with the ninth character in the text, which is 'b'. Based on the state transition definition, we move from node "aba" to node $\text{next}(\text{fail}(\text{"aba"}), 'b') = \text{"bab"}$. The node "bab" corresponds to one of the target patterns, so we record a match.



- Step 10: Continue with the tenth character in the text, which is 'a'. Based on the state transition definition, we move from node "bab" to node $\text{next}(\text{fail}(\text{"bab"}), 'a') = \text{"aba"}$. The node "aba" corresponds to one of the target patterns, so we record a match. Since node "aba" has a dictionary link pointing to node "ba", we also record the occurrence of the pattern "ba". Node "ba" has no further dictionary links, so no additional checks are needed. The algorithm terminates.



5.4. Complexity

5.4.1. Time complexity

- Let n be the length of the text T , m be the total length of all patterns, and z be the total number of occurrences of the patterns in T .
- Phase 1 (building the Trie): inserting all patterns into the Trie takes $O(m)$ time.
- Phase 2 (building the automaton, including failure links and dictionary links): we traverse all nodes of the Trie in BFS order, and for each node, we process all outgoing edges (i.e., characters in the alphabet). Therefore, the time complexity is $O(m * |\Sigma|)$.
- Phase 3 (matching): we scan the text T once. Additionally, each occurrence of a pattern takes $O(1)$ time to report. Hence, the time complexity is $O(n + z)$.
- Therefore, the overall time complexity of the Aho-Corasick algorithm is $O(m * |\Sigma| + n + z)$. In this report, Σ consists of lowercase English letters ('a'-'z'), so $|\Sigma| = 26$ is a constant. Thus, the time complexity can be considered as $O(m + n + z)$.

5.4.2. Space complexity

- To construct the Trie and the automaton, at most $m + 1$ nodes are required (m characters plus the root node). Each node stores information including the next array and the failure link, with size $|\Sigma|$. Therefore, the space complexity of the algorithm is $O(m * |\Sigma|)$, which can be considered $O(m)$ when $|\Sigma|$ is constant.

5.5. Discussion

- Since it is an extension of KMP, the Aho–Corasick algorithm also exhibits stable runtime performance. The best-case scenario occurs when none of the patterns appear in the text. In this case, building the Trie, constructing the automaton, and performing the matching all take $O(m + n)$ time, since $z = 0$.
- On the other hand, if the patterns are highly repetitive and appear frequently, for example "a", "aa", "aaa" in a text like "...aaaaaaaaaaaaaaaaaaaaa...", the algorithm incurs additional cost to report matches and follow dictionary links. The time complexity becomes $O(n + m + z)$, where z is large.
- Interestingly, when patterns are highly repetitive, the space may decrease, since the Trie requires fewer branches.

V. EXPERIMENTAL RESULTS

The data tables were calculated based on the average of three algorithm runtime measurements for each dataset (in milliseconds).

1. Random data

The random dataset consists of characters from 'a' to 'z' generated with random probability on a grid. This dataset possesses properties very similar to natural language (virtually no repetition, very few cycles).

1.1. Scenario 1 (Impact of Grid Size)

Scenario 1: Keyword sizes increase from 10x10 to 500x500 with a fixed number of 5 keywords.

Grid Size	N. Keywords	Brute-Force	Rabin-Karp	KMP	Boyer-Moore	Aho-corasick
10 x 10	5	0,053	0,080	0,140	0,100	0,110
50 x 50	5	0,817	1,117	1,860	0,880	0,463
100 x 100	5	3,410	4,367	7,290	1,860	1,343
500 x 500	5	85,250	123,580	176,650	16,913	35,277

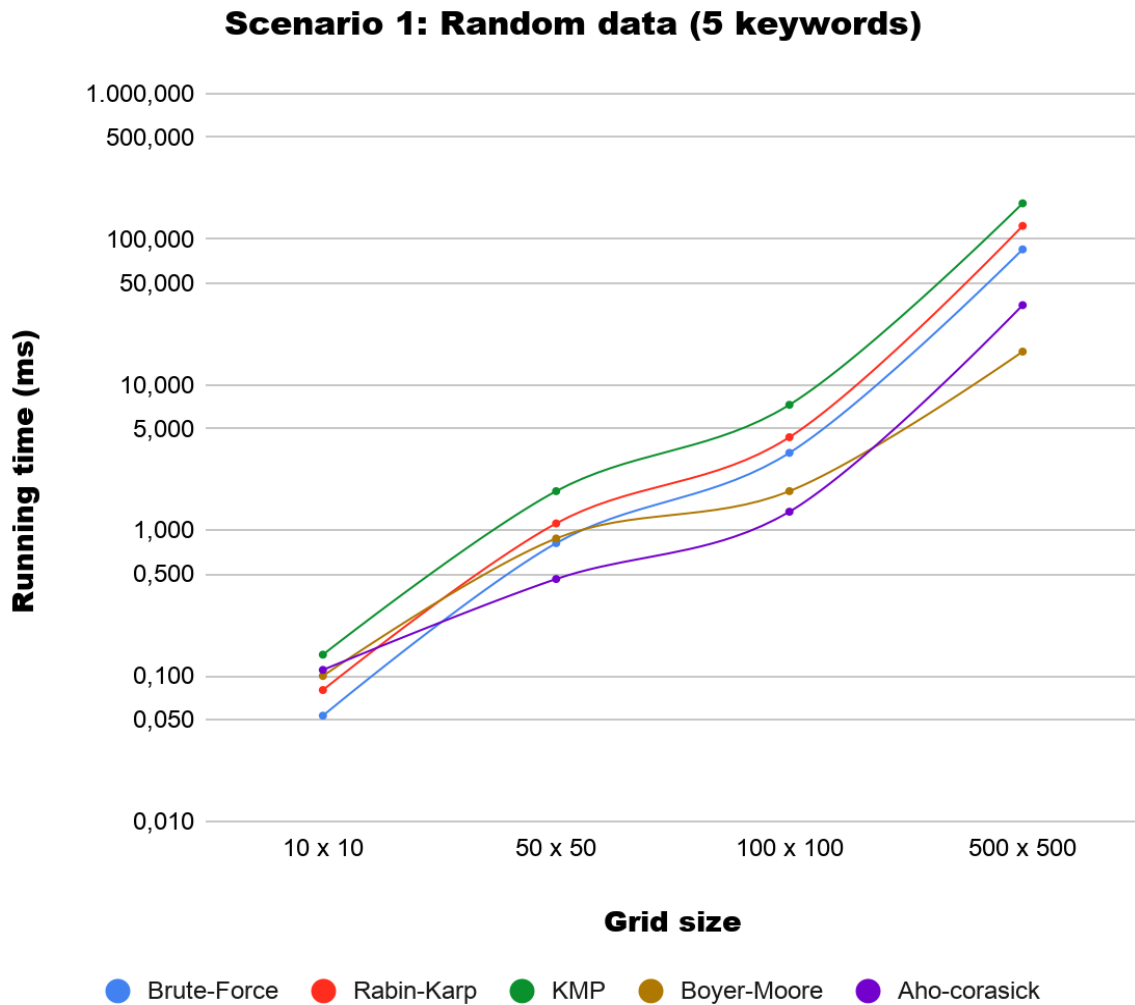


Chart 1.1: Runtime of random data across all algorithms in Scenario 1 (Impact of Grid Size)

The brute force algorithm demonstrates surprising performance; despite its theoretical quadratic time complexity, it exhibits linear growth. This is due to the uniform distribution of characters, which makes the probability of a match between two characters in the keyword and the grid extremely low. Consequently, brute force rarely encounters situations where it spends significant time matching only to backtrack upon a mismatch near the end of the pattern.

KMP and Rabin-Karp algorithms show linear growth in execution time, consistent with their theoretical first-order complexity. This is because KMP maintains a fixed complexity regardless of the input data characteristics. For Rabin-Karp, the nature of the dataset minimizes hash collisions, thereby reducing the overhead of re-verification.

Boyer-Moore and Aho-Corasick represent the fastest group of algorithms, though they operate via different mechanisms. Boyer-Moore benefits from its superior execution speed per pattern; with only 5 keywords, its performance remains top-tier. Meanwhile, Aho-Corasick is optimized for multi-pattern searching, so its execution speed with 5 keywords does not differ significantly from searching for a single keyword.

Overall, with a small number of keywords and a random dataset, all algorithms perform exceptionally well. However, when considering memory overhead and implementation complexity, the brute force algorithm is a noteworthy choice as it requires no auxiliary memory, is easy to understand, and can be implemented quickly.

1.2. Scenario 2 (Impact of Dictionary Size)

Scenario 2: The number of keywords increases from 10 to 100 with a fixed grid size of 100x100.

Grid Size	N. Keywords	Brute-Force	Rabin-Karp	KMP	Boyer-Moore	Aho-corasick
100 x 100	10	2,590	3,297	5,470	0,767	0,780
100 x 100	20	12,030	16,850	27,150	1,390	3,540
100 x 100	50	32,340	43,577	71,017	8,987	4,700
100 x 100	100	66,790	85,207	136,740	18,960	8,683

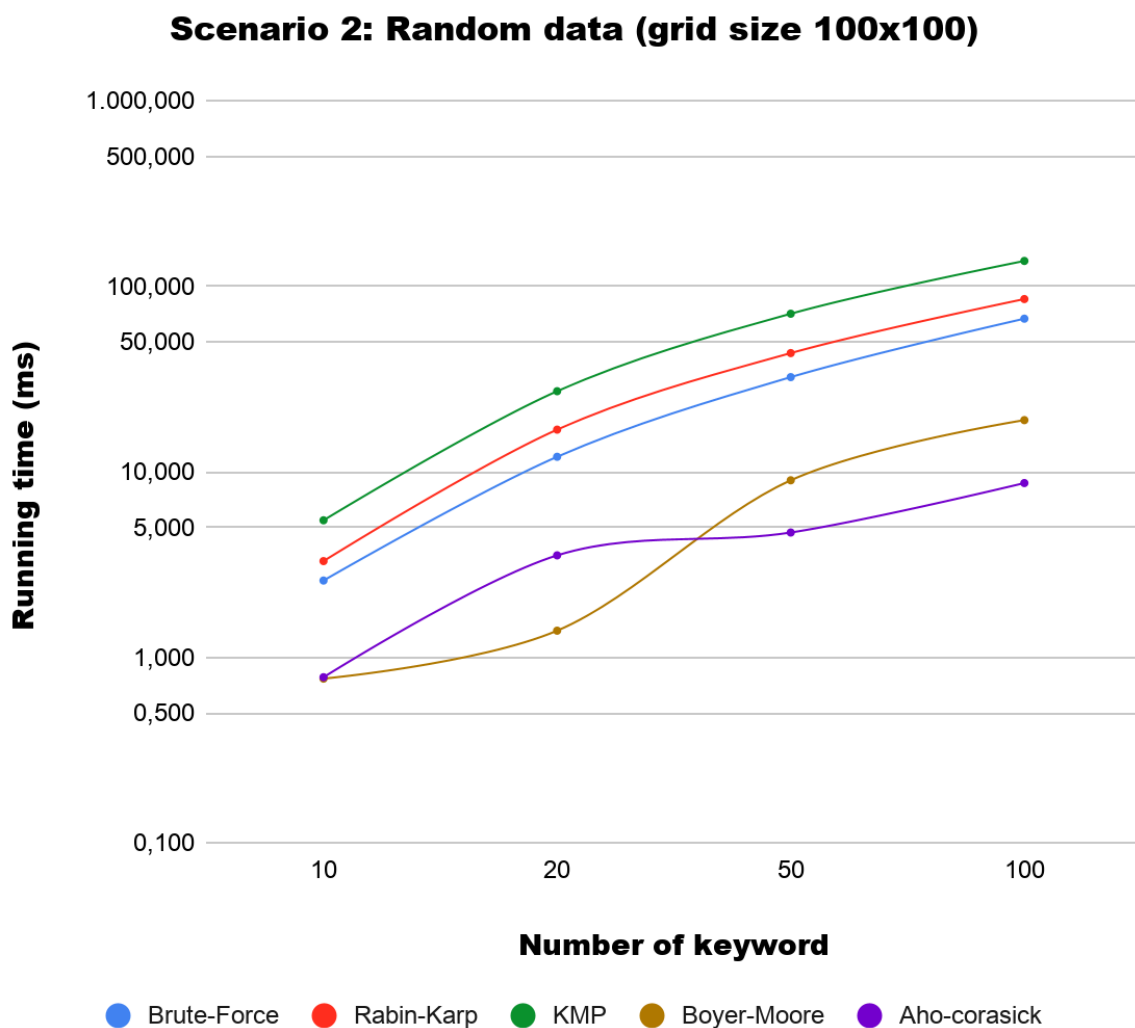


Chart 1.2: Runtime of random data across all algorithms in Scenario 2 (Impact of Dictionary Size)

As explained previously, due to the nature of the dataset, the execution speeds of the brute force, KMP, and Rabin-Karp algorithms all increase linearly with time. However, due to implementation details, there are minor differences in execution time between them. Brute force accesses memory sequentially and does not use external memory, making it highly optimized for hardware speed. Rabin-Karp utilizes the modulo operator (%), which

consumes dozens of times more CPU cycles than basic arithmetic operations (+-*/) continuously during the matching process, resulting in a slightly larger time constant. Additionally, in the source code, the KMP algorithm uses function pointers to access characters on the grid indirectly, which incurs extra overhead and leads to a larger time constant compared to the other algorithms.

The fastest group of algorithms remains Boyer-Moore and Aho-Corasick. The chart illustrates the strengths of these two algorithms, consistent with the analysis in Scenario 1. With a small number of keywords (10, 20), Boyer-Moore's speed is slightly superior to Aho-Corasick. However, as the number of keywords increases (50, 100), Boyer-Moore's advantage is no longer sufficient to maintain the top position. This is due to the relative size between the grid and the keywords. Since Boyer-Moore's strength lies in quickly skipping non-matching segments, datasets with larger grids and longer keywords can provide it with a further advantage. Conversely, the larger the number of keywords, the more clearly Aho-Corasick's speed advantage is demonstrated.

2. DNA data

The DNA dataset consists only of the characters 'A', 'C', 'G', and 'T', generated with random probability on a grid. Narrowing the alphabet size increases the probability of recurring patterns and repetitive characters.

2.1. Scenario 1 (Impact of Grid Size)

Scenario 1: Keyword sizes increase from 10x10 to 500x500 with a fixed number of 5 keywords.

Grid Size	N. Keywords	Brute-Force	Rabin-Karp	KMP	Boyer-Moore	Aho-corasick
10 x 10	5	0,080	0,090	0,150	0,110	0,123
50 x 50	5	1,400	1,463	2,493	2,063	0,690
100 x 100	5	5,437	4,843	8,940	6,450	2,140
500 x 500	5	139,583	101,853	136,087	93,587	33,670

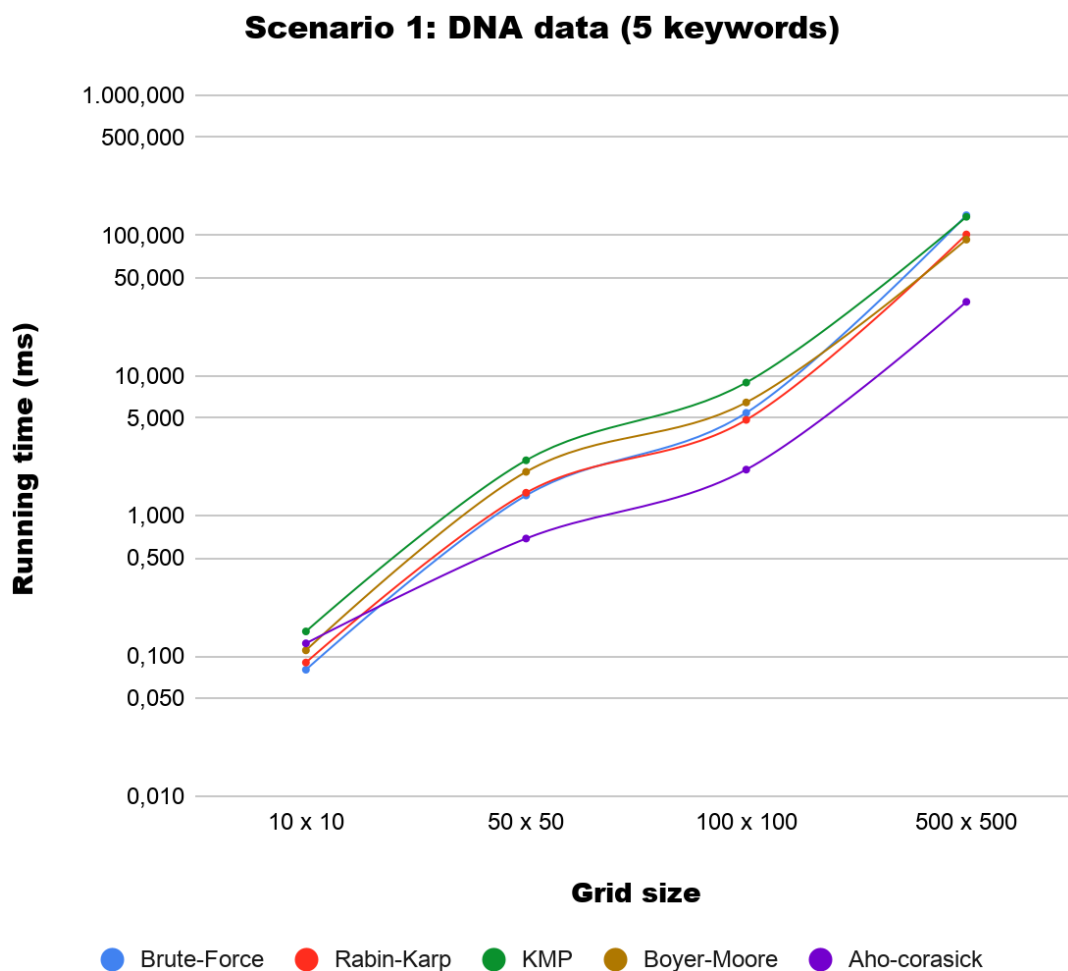


Chart 2.1: Runtime of DNA data across all algorithms in Scenario 1 (Impact of Grid Size)

As seen in the chart, the execution speeds of the algorithms still grow linearly over time. Among them, Aho-Corasick is the fastest, while the other algorithms have relatively similar running times.

For brute force, despite the small alphabet (only 4 characters: A, C, G, T), the probability of matching characters between the grid and the keyword remains very low (0.25^n , where n is the length of the matching sequence). Therefore, the running time of brute force remains a linear function with a small constant due to the algorithm's simplicity.

Similarly, with Rabin-Karp, the low overlap probability means the algorithm does not encounter many collision scenarios. Thus, its execution speed is also a linear function of time. KMP remains consistently stable and independent of the dataset characteristics, as previously analyzed.

However, for Boyer-Moore, the restricted alphabet makes the algorithm's jumps (based on the bad character rule) less effective than before, resulting in very short skip distances. Consequently, its execution speed approaches that of KMP and Rabin-Karp.

Aho-Corasick inherits the stability of KMP and further improves upon it using the Trie structure, making it less affected by the number of keywords. Therefore, despite having a complex tree structure using pointers, Aho-Corasick's speed remains the highest for this dataset.

2.2. Scenario 2 (Impact of Dictionary Size)

Scenario 2: The number of keywords increases from 10 to 100 with a fixed grid size of 100x100.

Grid Size	N. Keywords	Brute-Force	Rabin-Karp	KMP	Boyer-Moore	Aho-corasick
100 x 100	10	8,257	8,390	17,670	6,183	2,160
100 x 100	20	18,300	17,623	33,927	11,670	3,177
100 x 100	50	49,890	41,783	86,243	31,517	4,203
100 x 100	100	92,730	85,497	169,947	54,563	7,333

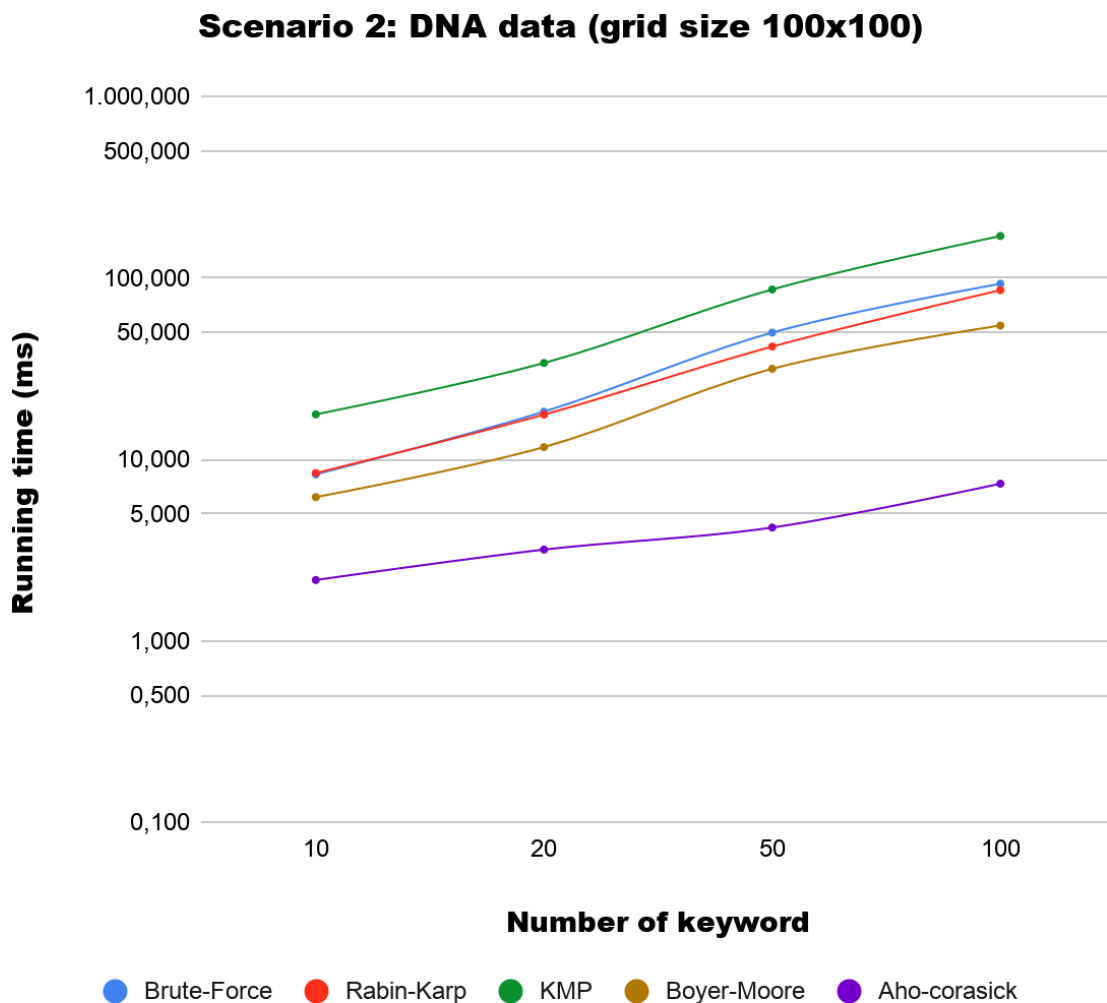


Chart 2.2: Runtime of DNA data across all algorithms in Scenario 2 (Impact of Dictionary Size)

The second scenario shares similar characteristics with the first; it merely amplifies the differences caused by the time constants and the number of keywords.

Among the algorithms, KMP is the slowest because its specific implementation has the largest time constant. Rabin-Karp and brute force utilize basic arithmetic operations, resulting in nearly identical execution speeds.

Although Boyer-Moore's bad character rule is not as powerful here as it is with a fully random dataset, repeating the process hundreds of times still creates a noticeable performance gap compared to the remaining single-pattern matching algorithms.

Aho-Corasick holds an absolute advantage due to the high number of keywords, making its execution speed significantly faster than all other algorithms.

3. Special data

The special dataset consists exclusively of the character 'a' for both the 100x100 grid and the 100 keywords.

Grid Size	N. Keywords	Brute-Force	Rabin-Karp	KMP	Boyer-Moore	Aho-corasick
100 x 100	100	894,063	1.121,190	221,550	1.406,990	142,820

The special dataset reveals an interesting phenomenon where the algorithms split into two distinct groups: highly efficient (Aho-Corasick, KMP) and inefficient (brute force, Rabin-Karp, and Boyer-Moore). To explain this, we must look back at the operational mechanics of the algorithms discussed in previous sections.

When the alphabet is reduced to only the character 'a', the repetition probability is 100%, forcing brute force to always match the entire keyword length. Similarly, for Rabin-Karp, the continuous appearance of keywords on the grid compels the algorithm to frequently perform brute-force re-verification to rule out hash collisions. The bad character and good suffix rules of Boyer-Moore also become completely ineffective in this dataset, essentially turning the algorithm into a much more cumbersome version of brute force.

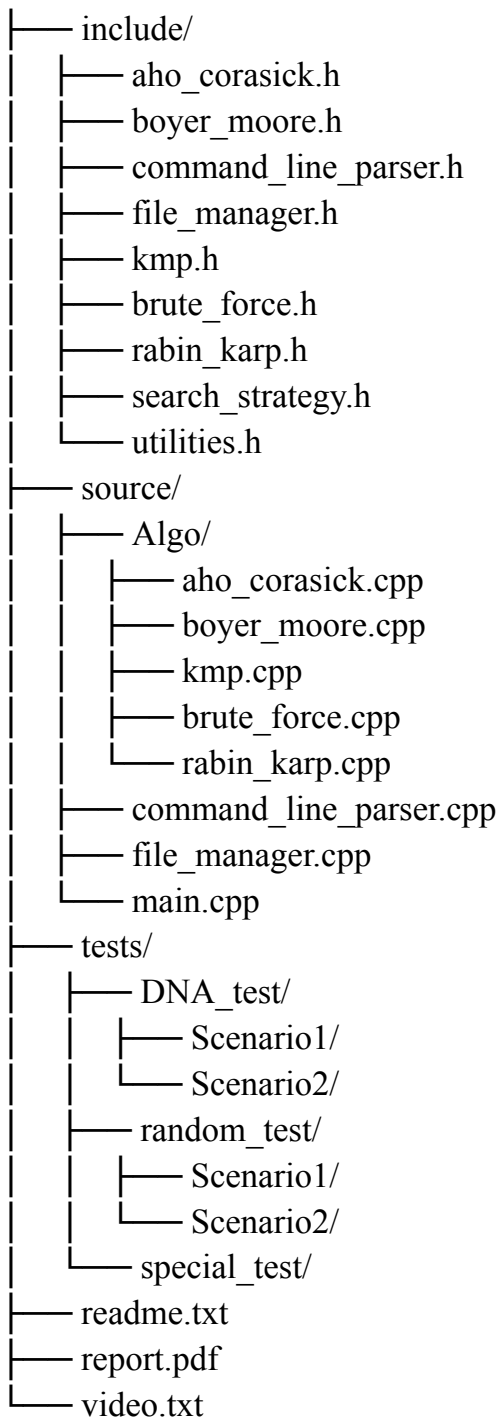
On the other hand, the exceptional stability of KMP and Aho-Corasick once again allows these two algorithms to maintain first-order linear execution time. This makes them significantly superior to the other three algorithms when encountering a special dataset composed entirely of 'a's. Aho-Corasick remains more optimal than KMP due to its inherent efficiency in multi-pattern searching.

VI. PROJECT ORGANIZATION AND PROGRAMMING NOTES

1. Project organization

The project consists of three main directories: the include folder containing header files (.h), the source folder containing source code files (.cpp), and the tests folder containing input files used to measure the execution time of the algorithms. Below is the detailed directory tree:

03/



The program's workflow begins with parsing the user's command line. First, the program validates the command line input; if it is invalid, an exception is returned, and the user is prompted to re-enter. Next, based on the user-provided values, the program reads data from the input file and selects the string processing algorithm requested by the user using the Strategy Pattern. Finally, the program processes the data and prints the results to an output file.

2. Compile and run environment

The benchmarks were performed on a team member's laptop. The results presented in the data table are the average of three consecutive measurements. The laptop configuration is as follows:

- CPU: AMD Ryzen AI 7 H 350w/Radeon 860M (2.00 GHz)
- RAM: 32.0 GB
- Operating system: Windows 11 Home
- Compiler: g++ 15.2.0
- C++ Standard: C++17

VII. REFERENCES

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to algorithms*, Fourth edition. Cambridge, Massachusetts: The MIT Press, 2022.
- [2] R. Sedgewick and K. D. Wayne, *Algorithms*, 4th ed. Upper Saddle River: Addison-Wesley, 2011.
- [3] “KMP Algorithm for Pattern Searching,” GeeksforGeeks. Accessed: Feb. 13, 2026. [Online]. Available: <https://www.geeksforgeeks.org/dsa/kmp-algorithm-for-pattern-searching/>
- [4] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, Massachusetts: The MIT Press, 2009.
- [5] R. Cole, “Tight Bounds on the Complexity of the Boyer–Moore String Matching Algorithm,” *SIAM J. Comput.*, vol. 23, no. 5, pp. 1075–1091, Oct. 1994, doi: 10.1137/S0097539791195543.
- [6] “Boyer Moore Algorithm: Fast String Searching Explained.” Accessed: Mar. 08, 2026. [Online]. Available: <https://www.ccbp.in/blog/articles/boyer-moore-algorithm>
- [7] “Trie Data Structure,” GeeksforGeeks. Accessed: Mar. 23, 2026. [Online]. Available: <https://www.geeksforgeeks.org/dsa/trie-insert-and-search/>
- [8] M. Crochemore and W. Rytter, *Jewels of stringology*. Singapore ; River Edge, NJ: World Scientific, 2002.
- [9] “Aho-Corasick Algorithm: Key Concepts, Code & Real Examples,” upGrad blog. Accessed: Mar. 23, 2026. [Online]. Available: <https://www.upgrad.com/blog/aho-corasick-algorithm/>

VIII. ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who has contributed to the completion of this project.

First and foremost, We extend our deepest appreciation to our subject teacher, Dr. Le Thanh Tung, Mr. Nguyen Thanh Tinh and Mr. Tran Hoang Quan, for unwavering support, giving us valuable guidance and pointing us in the right direction.

In addition, Google Gemini and ChatGPT, assisted with translating this report, searching documents.

We want to thank our fellow group members for their hard work, dedication, and cooperation throughout the project. Each team member played a crucial role, bringing unique skills and perspectives, greatly enriching our work.

Thank you all for making this project a fulfilling and rewarding experience.